



CHAPTER 4

CACHE MEMORY

4.1 Computer Memory System Overview

- Characteristics of Memory Systems
- The Memory Hierarchy

4.2 Cache Memory Principles

4.3 Elements of Cache Design

- Cache Addresses
- Cache Size
- Mapping Function
- Replacement Algorithms
- Write Policy
- Line Size
- Number of Caches

4.4 Pentium 4 Cache Organization

4.5 Key Terms, Review Questions, and Problems

Appendix 4A Performance Characteristics of Two-Level Memories

- Locality
- Operation of Two-Level Memory
- Performance

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- ◆ Present an overview of the main characteristics of computer memory systems and the use of a memory hierarchy.
- ◆ Describe the basic concepts and intent of cache memory.
- ◆ Discuss the key elements of cache design.
- ◆ Distinguish among direct mapping, associative mapping, and set-associative mapping.
- ◆ Explain the reasons for using multiple levels of cache.
- ◆ Understand the performance implications of multiple levels of memory.

Although seemingly simple in concept, computer memory exhibits perhaps the widest range of type, technology, organization, performance, and cost of any feature of a computer system. No single technology is optimal in satisfying the memory requirements for a computer system. As a consequence, the typical computer system is equipped with a hierarchy of memory subsystems, some internal to the system (directly accessible by the processor) and some external (accessible by the processor via an I/O module).

This chapter and the next focus on internal memory elements, while Chapter 6 is devoted to external memory. To begin, the first section examines key characteristics of computer memories. The remainder of the chapter examines an essential element of all modern computer systems: cache memory.

4.1 COMPUTER MEMORY SYSTEM OVERVIEW

Characteristics of Memory Systems

The complex subject of computer memory is made more manageable if we classify memory systems according to their key characteristics. The most important of these are listed in Table 4.1.

The term **location** in Table 4.1 refers to whether memory is internal or external to the computer. Internal memory is often equated with main memory, but there are other forms of internal memory. The processor requires its own local memory, in the form of registers (e.g., see Figure 2.3). Further, as we will see, the control unit portion of the processor may also require its own internal memory. We will defer discussion of these latter two types of internal memory to later chapters. Cache is another form of internal memory. External memory consists of peripheral storage devices, such as disk and tape, that are accessible to the processor via I/O controllers.

An obvious characteristic of memory is its **capacity**. For internal memory, this is typically expressed in terms of bytes (1 byte = 8 bits) or words. Common word lengths are 8, 16, and 32 bits. External memory capacity is typically expressed in terms of bytes.

Table 4.1 Key Characteristics of Computer Memory Systems

Location	Performance
Internal (e.g., processor registers, cache, main memory)	Access time
External (e.g., optical disks, magnetic disks, tapes)	Cycle time
	Transfer rate
Capacity	Physical Type
Number of words	Semiconductor
Number of bytes	Magnetic
	Optical
Unit of Transfer	Magneto-optical
Word	Physical Characteristics
Block	Volatile/nonvolatile
Access Method	Erasable/nonerasable
Sequential	Organization
Direct	Memory modules
Random	
Associative	

A related concept is the **unit of transfer**. For internal memory, the unit of transfer is equal to the number of electrical lines into and out of the memory module. This may be equal to the word length, but is often larger, such as 64, 128, or 256 bits. To clarify this point, consider three related concepts for internal memory:

- **Word:** The “natural” unit of organization of memory. The size of a word is typically equal to the number of bits used to represent an integer and to the instruction length. Unfortunately, there are many exceptions. For example, the CRAY C90 (an older model CRAY supercomputer) has a 64-bit word length but uses a 46-bit integer representation. The Intel x86 architecture has a wide variety of instruction lengths, expressed as multiples of bytes, and a word size of 32 bits.
- **Addressable units:** In some systems, the addressable unit is the word. However, many systems allow addressing at the byte level. In any case, the relationship between the length in bits A of an address and the number N of addressable units is $2^A = N$.
- **Unit of transfer:** For main memory, this is the number of bits read out of or written into memory at a time. The unit of transfer need not equal a word or an addressable unit. For external memory, data are often transferred in much larger units than a word, and these are referred to as blocks.

Another distinction among memory types is the **method of accessing** units of data. These include the following:

- **Sequential access:** Memory is organized into units of data, called records. Access must be made in a specific linear sequence. Stored addressing information is used to separate records and assist in the retrieval process. A shared read–write mechanism is used, and this must be moved from its current location to the desired location, passing and rejecting each intermediate record. Thus, the time to access an arbitrary record is highly variable. Tape units, discussed in Chapter 6, are sequential access.
- **Direct access:** As with sequential access, direct access involves a shared read–write mechanism. However, individual blocks or records have a unique

address based on physical location. Access is accomplished by direct access to reach a general vicinity plus sequential searching, counting, or waiting to reach the final location. Again, access time is variable. Disk units, discussed in Chapter 6, are direct access.

- **Random access:** Each addressable location in memory has a unique, physically wired-in addressing mechanism. The time to access a given location is independent of the sequence of prior accesses and is constant. Thus, any location can be selected at random and directly addressed and accessed. Main memory and some cache systems are random access.
- **Associative:** This is a random access type of memory that enables one to make a comparison of desired bit locations within a word for a specified match, and to do this for all words simultaneously. Thus, a word is retrieved based on a portion of its contents rather than its address. As with ordinary random-access memory, each location has its own addressing mechanism, and retrieval time is constant independent of location or prior access patterns. Cache memories may employ associative access.

From a user's point of view, the two most important characteristics of memory are capacity and **performance**. Three performance parameters are used:

- **Access time (latency):** For random-access memory, this is the time it takes to perform a read or write operation, that is, the time from the instant that an address is presented to the memory to the instant that data have been stored or made available for use. For non-random-access memory, access time is the time it takes to position the read–write mechanism at the desired location.
- **Memory cycle time:** This concept is primarily applied to random-access memory and consists of the access time plus any additional time required before a second access can commence. This additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively. Note that memory cycle time is concerned with the system bus, not the processor.
- **Transfer rate:** This is the rate at which data can be transferred into or out of a memory unit. For random-access memory, it is equal to $1/(\text{cycle time})$. For non-random-access memory, the following relationship holds:

$$T_n = T_A + \frac{n}{R} \quad (4.1)$$

where

T_n = Average time to read or write n bits

T_A = Average access time

n = Number of bits

R = Transfer rate, in bits per second (bps)

A variety of **physical types** of memory have been employed. The most common today are semiconductor memory, magnetic surface memory, used for disk and tape, and optical and magneto-optical.

Several **physical characteristics** of data storage are important. In a volatile memory, information decays naturally or is lost when electrical power is switched off. In a nonvolatile memory, information once recorded remains without deterioration until deliberately changed; no electrical power is needed to retain information. Magnetic-surface memories are nonvolatile. Semiconductor memory (memory on integrated circuits) may be either volatile or nonvolatile. Nonerasable memory cannot be altered, except by destroying the storage unit. Semiconductor memory of this type is known as *read-only memory* (ROM). Of necessity, a practical nonerasable memory must also be nonvolatile.

For random-access memory, the **organization** is a key design issue. In this context, *organization* refers to the physical arrangement of bits to form words. The obvious arrangement is not always used, as is explained in Chapter 5.

The Memory Hierarchy

The design constraints on a computer's memory can be summed up by three questions: How much? How fast? How expensive?

The question of how much is somewhat open ended. If the capacity is there, applications will likely be developed to use it. The question of how fast is, in a sense, easier to answer. To achieve greatest performance, the memory must be able to keep up with the processor. That is, as the processor is executing instructions, we would not want it to have to pause waiting for instructions or operands. The final question must also be considered. For a practical system, the cost of memory must be reasonable in relationship to other components.

As might be expected, there is a trade-off among the three key characteristics of memory: capacity, access time, and cost. A variety of technologies are used to implement memory systems, and across this spectrum of technologies, the following relationships hold:

- Faster access time, greater cost per bit;
- Greater capacity, smaller cost per bit;
- Greater capacity, slower access time.

The dilemma facing the designer is clear. The designer would like to use memory technologies that provide for large-capacity memory, both because the capacity is needed and because the cost per bit is low. However, to meet performance requirements, the designer needs to use expensive, relatively lower-capacity memories with short access times.

The way out of this dilemma is not to rely on a single memory component or technology, but to employ a **memory hierarchy**. A typical hierarchy is illustrated in Figure 4.1. As one goes down the hierarchy, the following occur:

- a. Decreasing cost per bit;
- b. Increasing capacity;
- c. Increasing access time;
- d. Decreasing frequency of access of the memory by the processor.

Thus, smaller, more expensive, faster memories are supplemented by larger, cheaper, slower memories. The key to the success of this organization

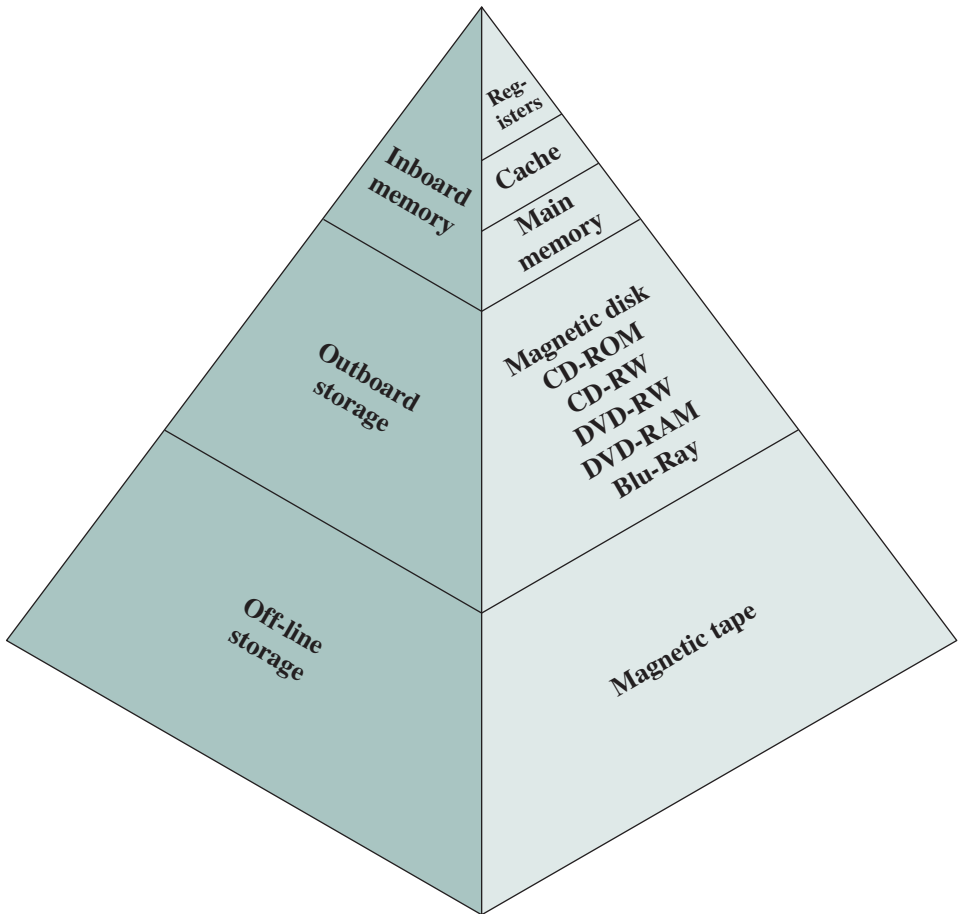


Figure 4.1 The Memory Hierarchy

is item (d): decreasing frequency of access. We examine this concept in greater detail when we discuss the cache, later in this chapter, and virtual memory in Chapter 8. A brief explanation is provided at this point.

The use of two levels of memory to reduce average access time works in principle, but only if conditions (a) through (d) apply. By employing a variety of technologies, a spectrum of memory systems exists that satisfies conditions (a) through (c). Fortunately, condition (d) is also generally valid.

The basis for the validity of condition (d) is a principle known as **locality of reference** [DENN68]. During the course of execution of a program, memory references by the processor, for both instructions and data, tend to cluster. Programs typically contain a number of iterative loops and subroutines. Once a loop or subroutine is entered, there are repeated references to a small set of instructions. Similarly, operations on tables and arrays involve access to a clustered set of data words. Over a long period of time, the clusters in use change, but over a short period of time, the processor is primarily working with fixed clusters of memory references.

EXAMPLE 4.1 Suppose that the processor has access to two levels of memory. Level 1 contains 1000 words and has an access time of $0.01 \mu s$; level 2 contains 100,000 words and has an access time of $0.1 \mu s$. Assume that if a word to be accessed is in level 1, then the processor accesses it directly. If it is in level 2, then the word is first transferred to level 1 and then accessed by the processor. For simplicity, we ignore the time required for the processor to determine whether the word is in level 1 or level 2. Figure 4.2 shows the general shape of the curve that covers this situation. The figure shows the average access time to a two-level memory as a function of the hit ratio H , where H is defined as the fraction of all memory accesses that are found in the faster memory (e.g., the cache), T_1 is the access time to level 1, and T_2 is the access time to level 2.¹ As can be seen, for high percentages of level 1 access, the average total access time is much closer to that of level 1 than that of level 2.

In our example, suppose 95% of the memory accesses are found in level 1. Then the average time to access a word can be expressed as

$$(0.95)(0.01 \mu s) + (0.05)(0.01 \mu s + 0.1 \mu s) = 0.0095 + 0.0055 = 0.015 \mu s$$

The average access time is much closer to $0.01 \mu s$ than to $0.1 \mu s$, as desired.

Accordingly, it is possible to organize data across the hierarchy such that the percentage of accesses to each successively lower level is substantially less than that of the level above. Consider the two-level example already presented. Let level 2

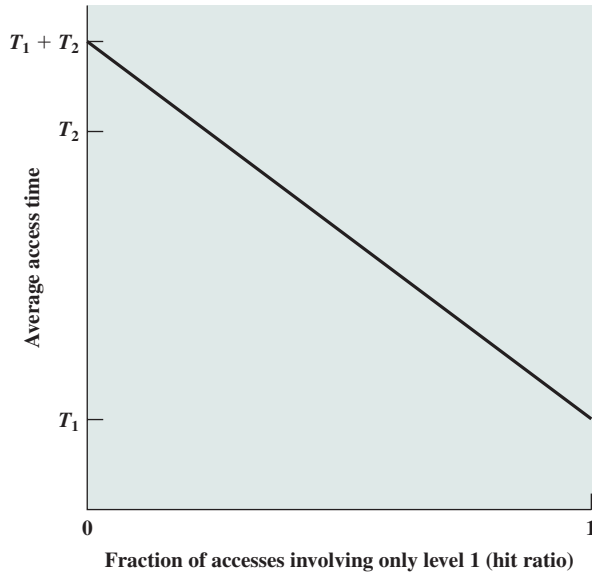


Figure 4.2 Performance of Accesses Involving only Level 1 (hit ratio)

¹If the accessed word is found in the faster memory, that is defined as a **hit**. A **miss** occurs if the accessed word is not found in the faster memory.

memory contain all program instructions and data. The current clusters can be temporarily placed in level 1. From time to time, one of the clusters in level 1 will have to be swapped back to level 2 to make room for a new cluster coming in to level 1. On average, however, most references will be to instructions and data contained in level 1.

This principle can be applied across more than two levels of memory, as suggested by the hierarchy shown in Figure 4.1. The fastest, smallest, and most expensive type of memory consists of the registers internal to the processor. Typically, a processor will contain a few dozen such registers, although some machines contain hundreds of registers. Main memory is the principal internal memory system of the computer. Each location in main memory has a unique address. Main memory is usually extended with a higher-speed, smaller cache. The cache is not usually visible to the programmer or, indeed, to the processor. It is a device for staging the movement of data between main memory and processor registers to improve performance.

The three forms of memory just described are, typically, volatile and employ semiconductor technology. The use of three levels exploits the fact that semiconductor memory comes in a variety of types, which differ in speed and cost. Data are stored more permanently on external mass storage devices, of which the most common are hard disk and removable media, such as removable magnetic disk, tape, and optical storage. External, nonvolatile memory is also referred to as **secondary memory** or **auxiliary memory**. These are used to store program and data files and are usually visible to the programmer only in terms of files and records, as opposed to individual bytes or words. Disk is also used to provide an extension to main memory known as virtual memory, which is discussed in Chapter 8.

Other forms of memory may be included in the hierarchy. For example, large IBM mainframes include a form of internal memory known as expanded storage. This uses a semiconductor technology that is slower and less expensive than that of main memory. Strictly speaking, this memory does not fit into the hierarchy but is a side branch: Data can be moved between main memory and expanded storage but not between expanded storage and external memory. Other forms of secondary memory include optical and magneto-optical disks. Finally, additional levels can be effectively added to the hierarchy in software. A portion of main memory can be used as a buffer to hold data temporarily that is to be read out to disk. Such a technique, sometimes referred to as a disk cache,² improves performance in two ways:

- Disk writes are clustered. Instead of many small transfers of data, we have a few large transfers of data. This improves disk performance and minimizes processor involvement.
- Some data destined for write-out may be referenced by a program before the next dump to disk. In that case, the data are retrieved rapidly from the software cache rather than slowly from the disk.

Appendix 4A examines the performance implications of multilevel memory structures.

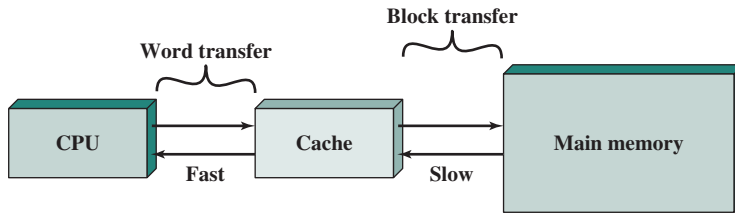
² Disk cache is generally a purely software technique and is not examined in this book. See [STAL15] for a discussion.

4.2 CACHE MEMORY PRINCIPLES

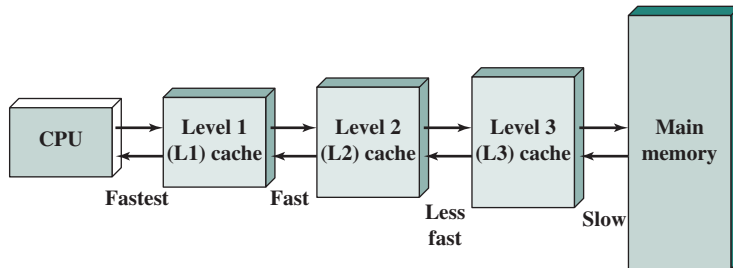
Cache memory is designed to combine the memory access time of expensive, high-speed memory combined with the large memory size of less expensive, lower-speed memory. The concept is illustrated in Figure 4.3a. There is a relatively large and slow main memory together with a smaller, faster cache memory. The cache contains a copy of portions of main memory. When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache. If so, the word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block.

Figure 4.3b depicts the use of multiple levels of cache. The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache.

Figure 4.4 depicts the structure of a cache/main-memory system. Main memory consists of up to 2^n addressable words, with each word having a unique n -bit address. For mapping purposes, this memory is considered to consist of a number of fixed-length blocks of K words each. That is, there are $M = 2^n/K$ blocks in main memory. The cache consists of m blocks, called **lines**.³ Each line contains K words,



(a) Single cache



(b) Three-level cache organization

Figure 4.3 Cache and Main Memory

³In referring to the basic unit of the cache, the term *line* is used, rather than the term *block*, for two reasons: (1) to avoid confusion with a main memory block, which contains the same number of data words as a cache line; and (2) because a cache line includes not only K words of data, just as a main memory block, but also includes tag and control bits.

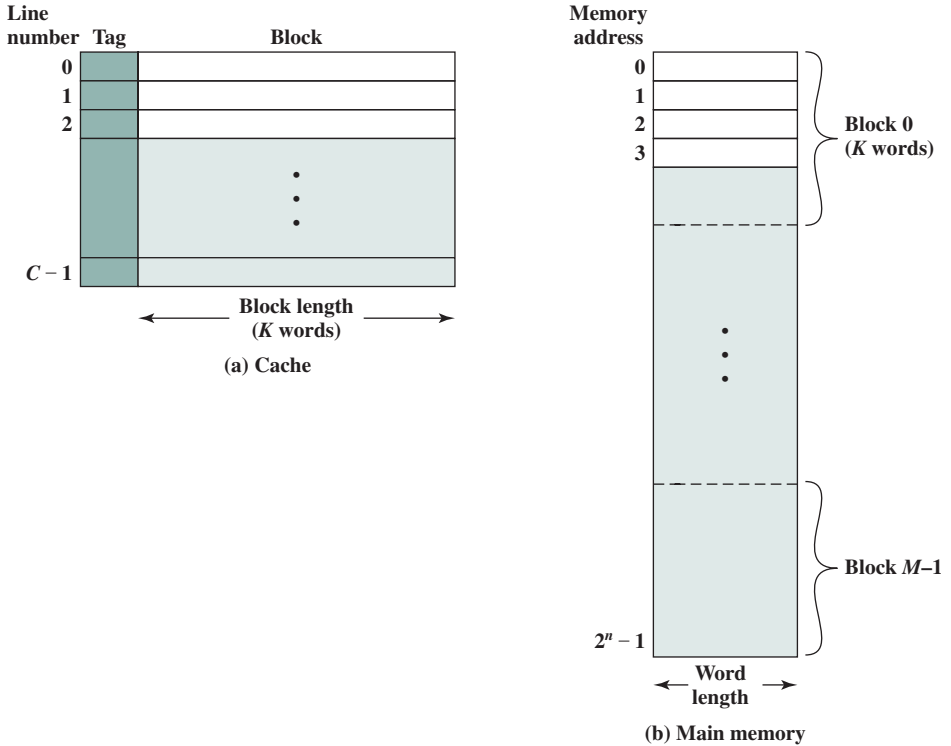


Figure 4.4 Cache/Main Memory Structure

plus a tag of a few bits. Each line also includes control bits (not shown), such as a bit to indicate whether the line has been modified since being loaded into the cache. The length of a line, not including tag and control bits, is the **line size**. The line size may be as small as 32 bits, with each “word” being a single byte; in this case the line size is 4 bytes. The number of lines is considerably less than the number of main memory blocks ($m \ll M$). At any time, some subset of the blocks of memory resides in lines in the cache. If a word in a block of memory is read, that block is transferred to one of the lines of the cache. Because there are more blocks than lines, an individual line cannot be uniquely and permanently dedicated to a particular block. Thus, each line includes a **tag** that identifies which particular block is currently being stored. The tag is usually a portion of the main memory address, as described later in this section.

Figure 4.5 illustrates the read operation. The processor generates the read address (RA) of a word to be read. If the word is contained in the cache, it is delivered to the processor. Otherwise, the block containing that word is loaded into the cache, and the word is delivered to the processor. Figure 4.5 shows these last two operations occurring in parallel and reflects the organization shown in Figure 4.6, which is typical of contemporary cache organizations. In this organization, the cache connects to the processor via data, control, and address lines. The data and address lines also attach to data and address buffers, which attach to a system bus from

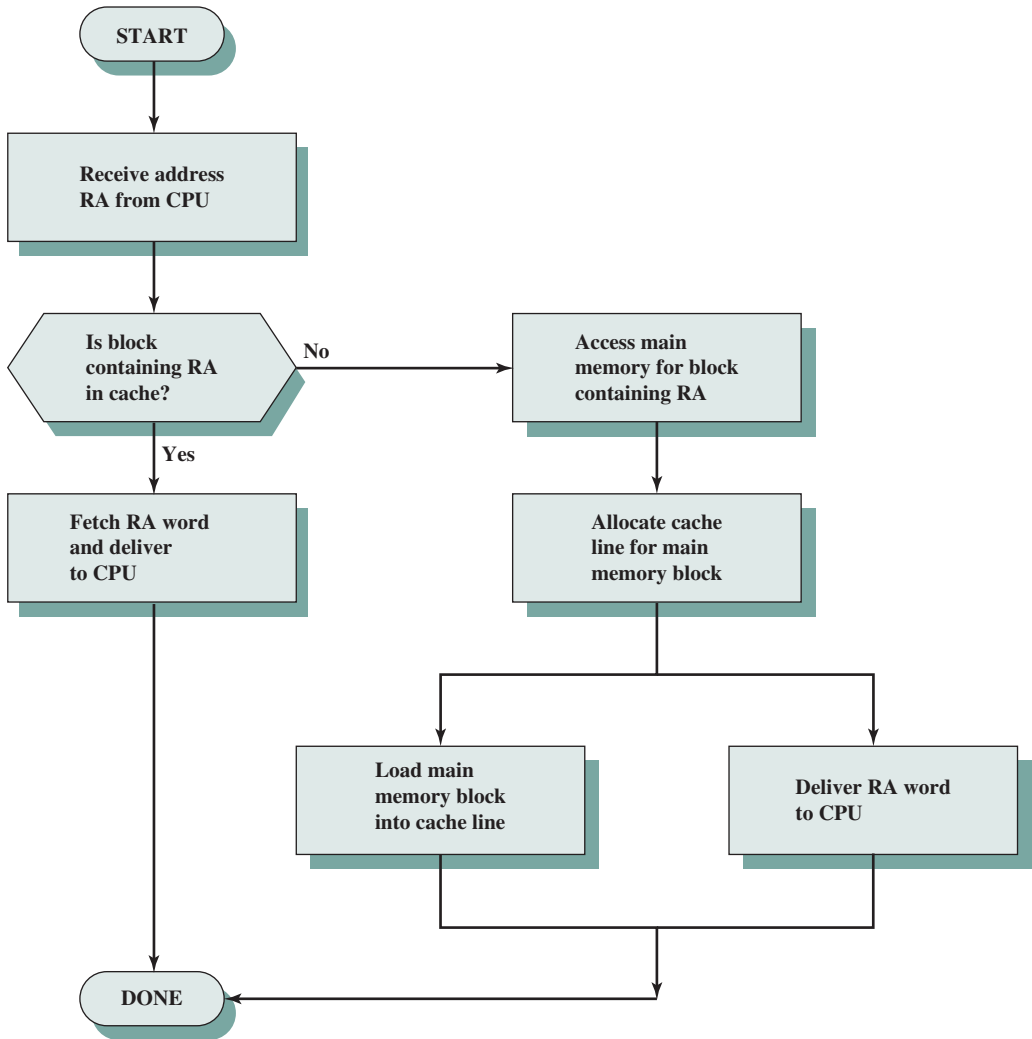


Figure 4.5 Cache Read Operation

which main memory is reached. When a cache hit occurs, the data and address buffers are disabled and communication is only between processor and cache, with no system bus traffic. When a cache miss occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to both the cache and the processor. In other organizations, the cache is physically interposed between the processor and the main memory for all data, address, and control lines. In this latter case, for a cache miss, the desired word is first read into the cache and then transferred from cache to processor.

A discussion of the performance parameters related to cache use is contained in Appendix 4A.

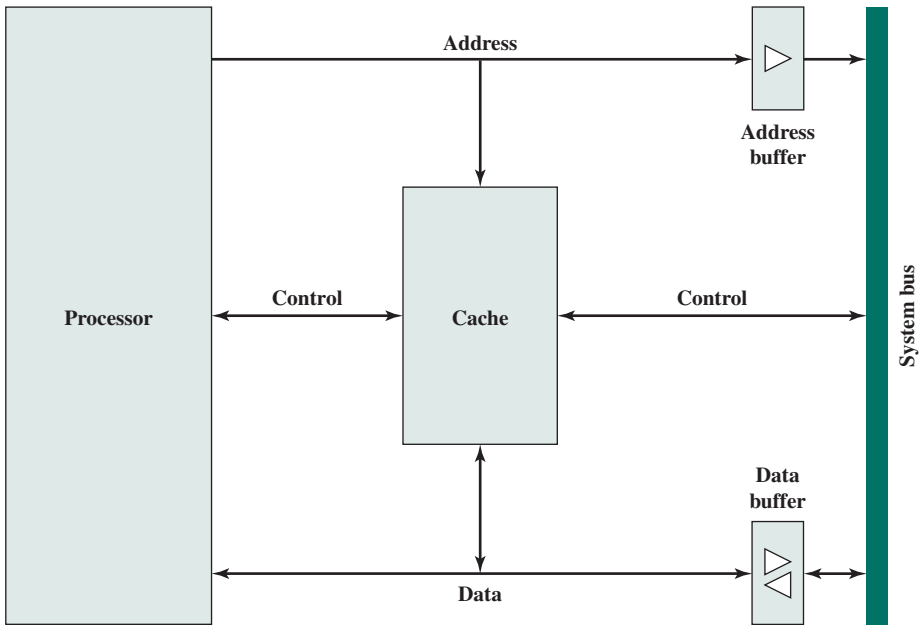


Figure 4.6 Typical Cache Organization

4.3 ELEMENTS OF CACHE DESIGN

This section provides an overview of cache design parameters and reports some typical results. We occasionally refer to the use of caches in **high-performance computing (HPC)**. HPC deals with supercomputers and their software, especially for scientific applications that involve large amounts of data, vector and matrix computation, and the use of parallel algorithms. Cache design for HPC is quite different than for other hardware platforms and applications. Indeed, many researchers have found that HPC applications perform poorly on computer architectures that employ caches [BAIL93]. Other researchers have since shown that a cache hierarchy can be useful in improving performance if the application software is tuned to exploit the cache [WANG99, PRES01].⁴

Although there are a large number of cache implementations, there are a few basic design elements that serve to classify and differentiate cache architectures. Table 4.2 lists key elements.

Cache Addresses

Almost all nonembedded processors, and many embedded processors, support virtual memory, a concept discussed in Chapter 8. In essence, virtual memory is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available. When virtual memory is used, the address fields of machine instructions contain virtual addresses. For reads

⁴For a general discussion of HPC, see [DOWD98].

Table 4.2 Elements of Cache Design

Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Line Size
Mapping Function	Number of Caches
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

to and writes from main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory.

When virtual addresses are used, the system designer may choose to place the cache between the processor and the MMU or between the MMU and main memory (Figure 4.7). A **logical cache**, also known as a **virtual cache**, stores data using

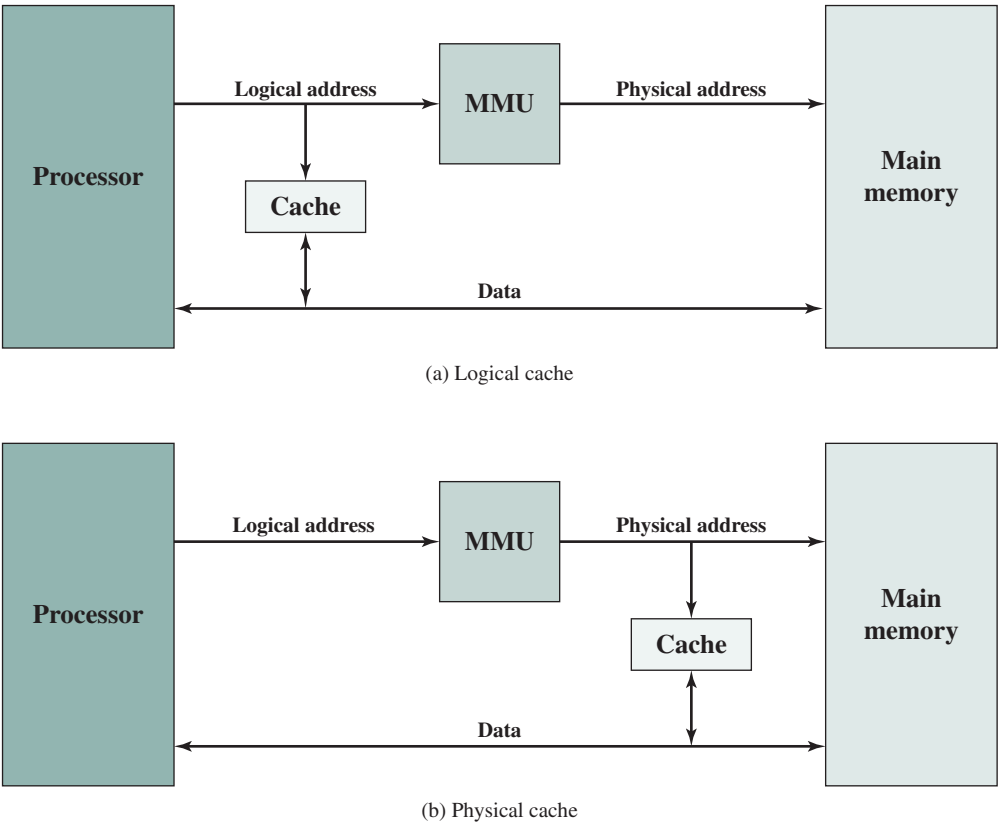


Figure 4.7 Logical and Physical Caches

virtual addresses. The processor accesses the cache directly, without going through the MMU. A **physical cache** stores data using main memory **physical addresses**.

One obvious advantage of the logical cache is that cache access speed is faster than for a physical cache, because the cache can respond before the MMU performs an address translation. The disadvantage has to do with the fact that most virtual memory systems supply each application with the same virtual memory address space. That is, each application sees a virtual memory that starts at address 0. Thus, the same virtual address in two different applications refers to two different physical addresses. The cache memory must therefore be completely flushed with each application context switch, or extra bits must be added to each line of the cache to identify which virtual address space this address refers to.

The subject of logical versus physical cache is a complex one, and beyond the scope of this book. For a more in-depth discussion, see [CEKL97] and [JACO08].

Cache Size

The second item in Table 4.2, cache size, has already been discussed. We would like the size of the cache to be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone. There are several other motivations for minimizing cache size. The larger the cache, the larger the number of gates involved in addressing the cache. The result is that large caches tend to be slightly slower than small ones—even when built with the same integrated circuit technology and put in the same place on chip and circuit board. The available chip and board area also limits cache size. Because the performance of the cache is very sensitive to the nature of the workload, it is impossible to arrive at a single “optimum” cache size. Table 4.3 lists the cache sizes of some current and past processors.

Mapping Function

Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines. Further, a means is needed for determining which main memory block currently occupies a cache line. The choice of the mapping function dictates how the cache is organized. Three techniques can be used: direct, associative, and set-associative. We examine each of these in turn. In each case, we look at the general structure and then a specific example.

EXAMPLE 4.2 For all three cases, the example includes the following elements:

- The cache can hold 64 kB.
- Data are transferred between main memory and the cache in blocks of 4 bytes each. This means that the cache is organized as $16K = 2^{14}$ lines of 4 bytes each.
- The main memory consists of 16 MB, with each byte directly addressable by a 24-bit address ($2^{24} = 16M$). Thus, for mapping purposes, we can consider main memory to consist of 4M blocks of 4 bytes each.

Table 4.3 Cache Sizes of Some Processors

Processor	Type	Year of Introduction	L1 Cache ^a	L2 Cache	L3 Cache
IBM 360/85	Mainframe	1968	16–32 kB	—	—
PDP-11/70	Minicomputer	1975	1 kB	—	—
VAX 11/780	Minicomputer	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128–256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256–512 kB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/server	1999	32 kB/32 kB	256 kB to 1 MB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/server	2000	8 kB/8 kB	256 kB	—
IBM SP	High-end server/ supercomputer	2000	64 kB/32 kB	8 MB	—
CRAY MTA ^b	Supercomputer	2000	8 kB	2 MB	—
Itanium	PC/server	2001	16 kB/16 kB	96 kB	4 MB
Itanium 2	PC/server	2002	32 kB	256 kB	6 MB
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1 MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24–48 MB
Intel Core i7 EE 990	Workstation/ server	2011	6 × 32 kB/ 32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/ server	2011	24 × 64 kB/ 128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

Notes: ^a Two values separated by a slash refer to instruction and data caches. ^b Both caches are instruction only; no data caches.

DIRECT MAPPING The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line. The mapping is expressed as

$$i = j \text{ modulo } m$$

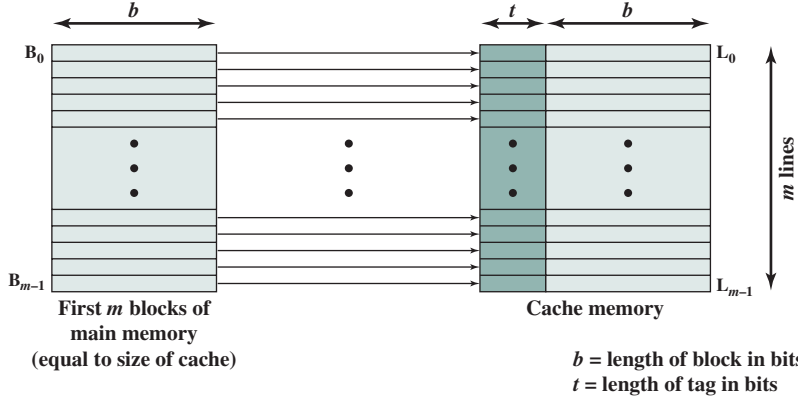
where

i = cache line number

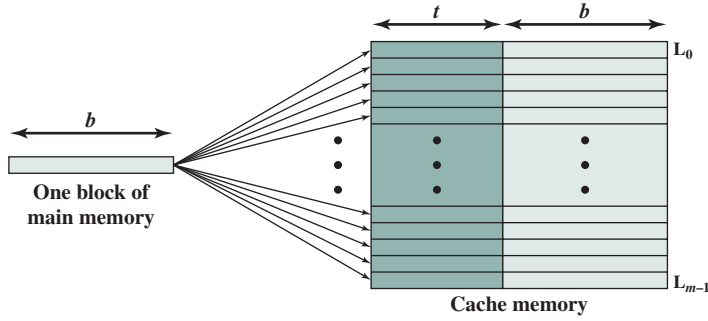
j = main memory block number

m = number of lines in the cache

Figure 4.8a shows the mapping for the first m blocks of main memory. Each block of main memory maps into one unique line of the cache. The next m blocks



(a) Direct mapping



(b) Associative mapping

Figure 4.8 Mapping from Main Memory to Cache: Direct and Associative

of main memory map into the cache in the same fashion; that is, block B_m of main memory maps into line L_0 of cache, block B_{m+1} maps into line L_1 , and so on.

The mapping function is easily implemented using the main memory address. Figure 4.9 illustrates the general mechanism. For purposes of cache access, each main memory address can be viewed as consisting of three fields. The least significant w bits identify a unique word or byte within a block of main memory; in most contemporary machines, the address is at the byte level. The remaining s bits specify one of the 2^s blocks of main memory. The cache logic interprets these s bits as a tag of $s - r$ bits (most significant portion) and a line field of r bits. This latter field identifies one of the $m = 2^r$ lines of the cache. To summarize,

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $\frac{2^{s+w}}{2^w} = 2^s$
- Number of lines in cache = $m = 2^r$
- Size of cache = 2^{r+w} words or bytes
- Size of tag = $(s - r)$ bits

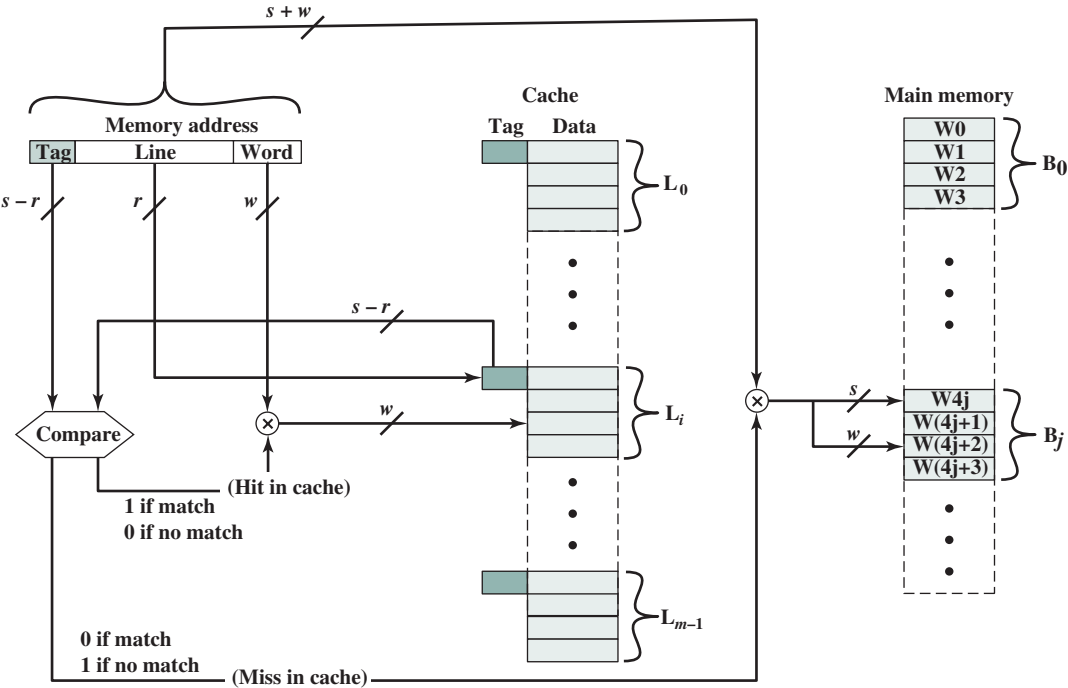


Figure 4.9 Direct-Mapping Cache Organization

EXAMPLE 4.2a Figure 4.10 shows our example system using direct mapping.⁵ In the example, $m = 16K = 2^{14}$ and $i = j$ modulo 2^{14} . The mapping becomes

Cache Line	Starting Memory Address of Block
0	000000, 010000, ..., FF0000
1	000004, 010004, ..., FF0004
$\vdots$	$\vdots$
$2^{14} - 1$	00FFFC, 01FFFC, ..., FFFFFC

Note that no two blocks that map into the same line number have the same tag number. Thus, blocks with starting addresses 000000, 010000, ..., FF0000 have tag numbers 00, 01, ..., FF, respectively. Referring back to Figure 4.5, a read operation works as follows. The cache system is presented with a 24-bit address. The 14-bit line number is used as an index into the cache to access a particular line. If the 8-bit tag number matches the tag number currently stored in that line, then the 2-bit word number is used to select one of the 4 bytes in that line. Otherwise, the 22-bit tag-plus-line field is used to fetch a block from main memory. The actual address that is used for the fetch is the 22-bit tag-plus-line concatenated with two 0 bits, so that 4 bytes are fetched starting on a block boundary.

⁵In this and subsequent figures, memory values are represented in hexadecimal notation. See Chapter 9 for a basic refresher on number systems (decimal, binary, hexadecimal).

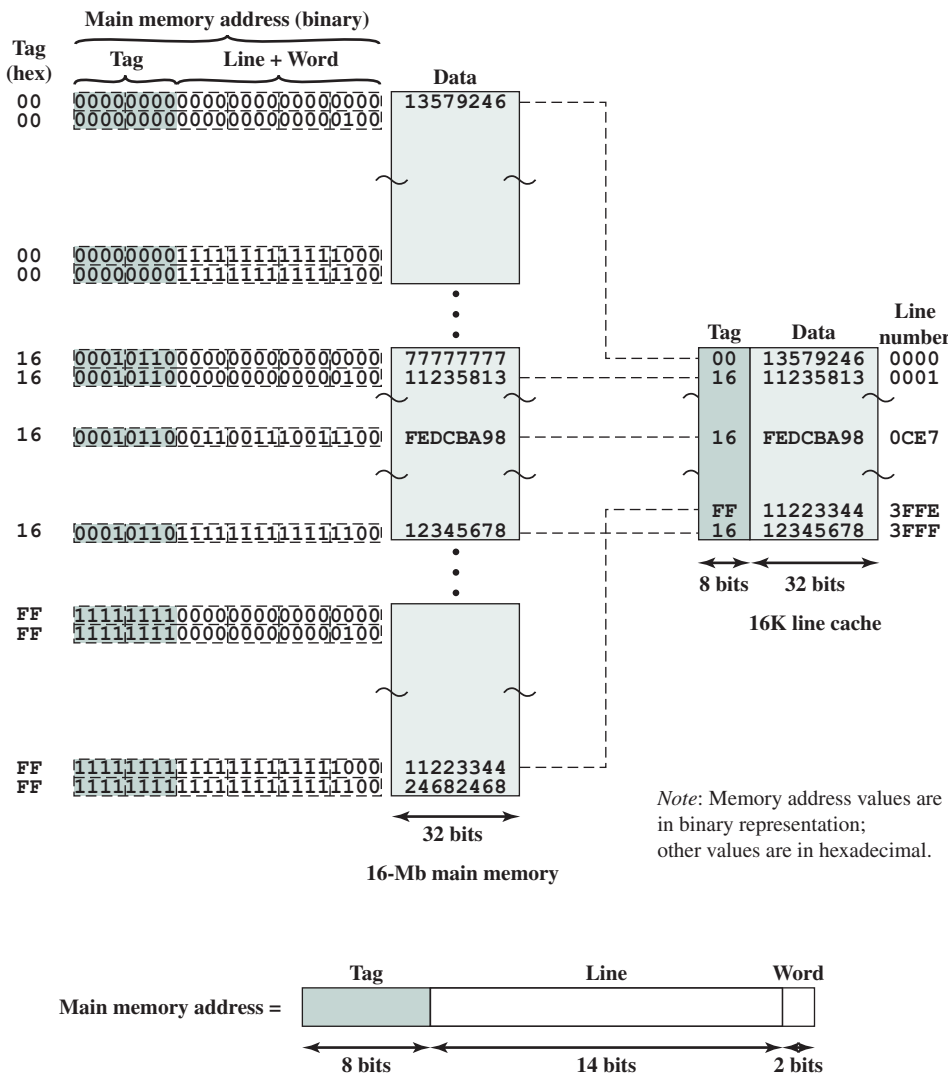


Figure 4.10 Direct Mapping Example

The effect of this mapping is that blocks of main memory are assigned to lines of the cache as follows:

Cache line	Main memory blocks assigned
0	$0, m, 2m, \dots, 2^s - m$
1	$1, m + 1, 2m + 1, \dots, 2^s - m + 1$
$\vdots$	$\vdots$
$m - 1$	$m - 1, 2m - 1, 3m - 1, \dots, 2^s - 1$

Thus, the use of a portion of the address as a line number provides a unique mapping of each block of main memory into the cache. When a block is actually

read into its assigned line, it is necessary to tag the data to distinguish it from other blocks that can fit into that line. The most significant $s - r$ bits serve this purpose.

The direct mapping technique is simple and inexpensive to implement. Its main disadvantage is that there is a fixed cache location for any given block. Thus, if a program happens to reference words repeatedly from two different blocks that map into the same line, then the blocks will be continually swapped in the cache, and the hit ratio will be low (a phenomenon known as *thrashing*).



Selective Victim Cache Simulator

One approach to lower the **miss** penalty is to remember what was discarded in case it is needed again. Since the discarded data has already been fetched, it can be used again at a small cost. Such recycling is possible using a victim cache. Victim cache was originally proposed as an approach to reduce the conflict misses of direct mapped caches without affecting its fast access time. Victim cache is a fully associative cache, whose size is typically 4 to 16 cache lines, residing between a direct mapped L1 cache and the next level of memory. This concept is explored in Appendix F.

ASSOCIATIVE MAPPING Associative mapping overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of the cache (Figure 4.8b). In this case, the cache control logic interprets a memory address simply as a Tag and a Word field. The Tag field uniquely identifies a block of main memory. To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's tag for a match. Figure 4.11 illustrates the logic.

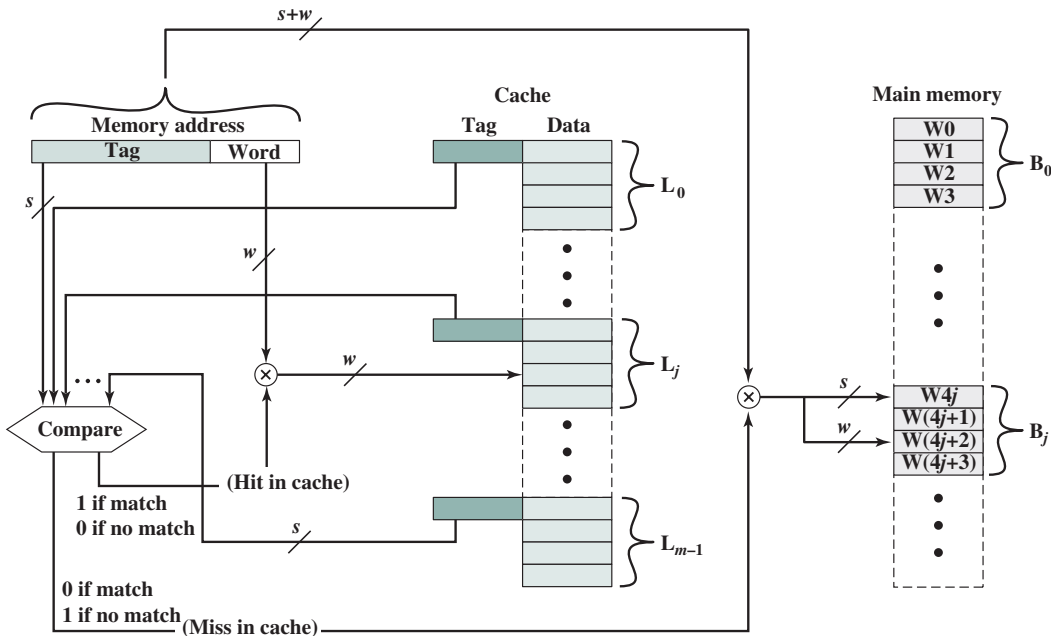


Figure 4.11 Fully Associative Cache Organization

EXAMPLE 4.2b Figure 4.12 shows our example using associative mapping. A main memory address consists of a 22-bit tag and a 2-bit byte number. The 22-bit tag must be stored with the 32-bit block of data for each line in the cache. Note that it is the leftmost (most significant) 22 bits of the address that form the tag. Thus, the 24-bit hexadecimal address 16339C has the 22-bit tag 058CE7. This is easily seen in binary notation:

Memory address	0001	0110	0011	0011	1001	1100	(binary)
	1	6	3	3	9	C	(hex)
Tag (leftmost 22 bits)	00	0101	1000	1100	1110	0111	(binary)
	0	5	8	C	E	7	(hex)

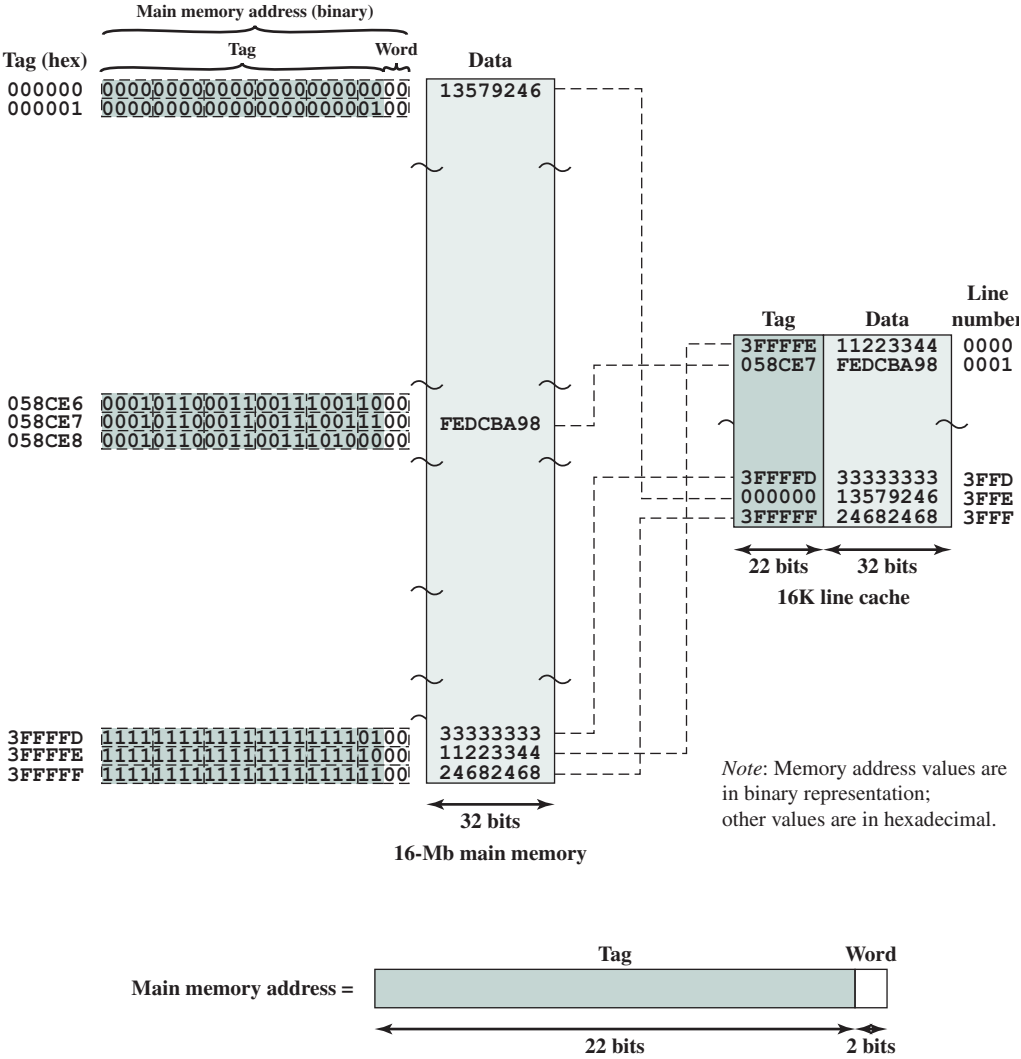


Figure 4.12 Associative Mapping Example

Note that no field in the address corresponds to the line number, so that the number of lines in the cache is not determined by the address format. To summarize,

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $\frac{2^{s+w}}{2^w} = 2^s$
- Number of lines in cache = undetermined
- Size of tag = s bits

With associative mapping, there is flexibility as to which block to replace when a new block is read into the cache. Replacement algorithms, discussed later in this section, are designed to maximize the hit ratio. The principal disadvantage of associative mapping is the complex circuitry required to examine the tags of all cache lines in parallel.



Cache Time Analysis Simulator

SET-ASSOCIATIVE MAPPING Set-associative mapping is a compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages.

In this case, the cache consists of number sets, each of which consists of a number of lines. The relationships are

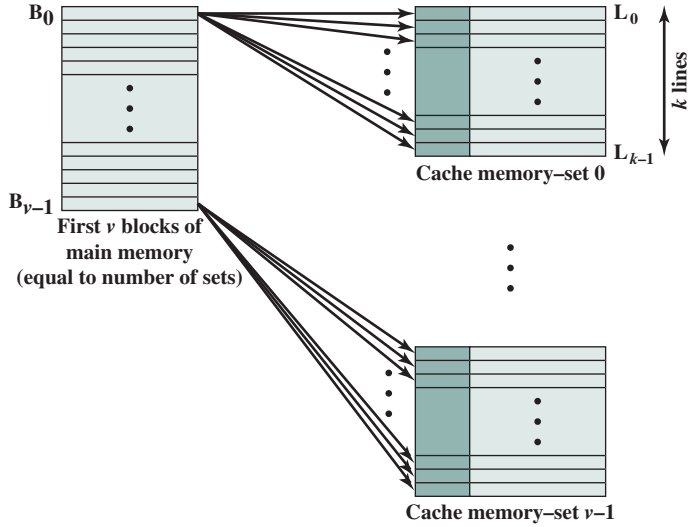
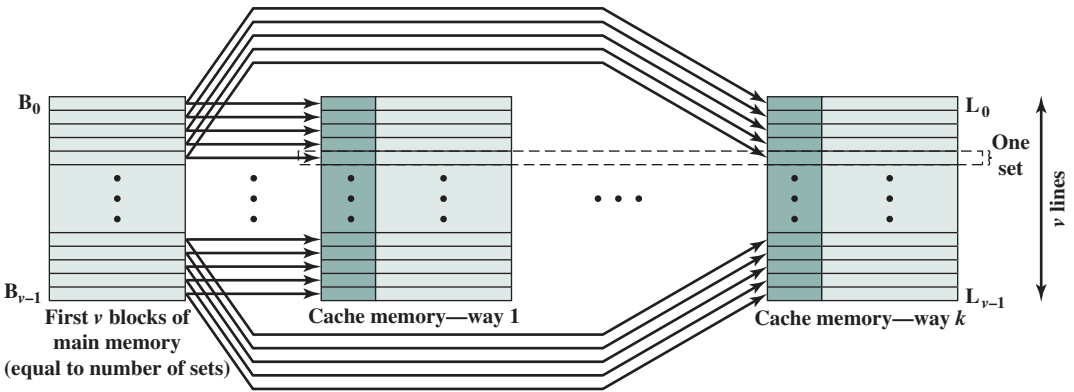
$$m = v \times k$$

$$i = j \text{ modulo } v$$

where

- i = cache set number
- j = main memory block number
- m = number of lines in the cache
- v = number of sets
- k = number of lines in each set

This is referred to as k -way set-associative mapping. With set-associative mapping, block B_j can be mapped into any of the lines of set j . Figure 4.13a illustrates this mapping for the first v blocks of main memory. As with associative mapping, each word maps into multiple cache lines. For set-associative mapping, each word maps into all the cache lines in a specific set, so that main memory block B_0 maps into set 0, and so on. Thus, the set-associative cache can be physically implemented as v associative caches. It is also possible to implement the set-associative cache as k direct mapping caches, as shown in Figure 4.13b. Each direct-mapped cache is referred to as a *way*, consisting of v lines. The first v lines of main memory are direct mapped into the v lines of each way; the next group of v lines of main memory are similarly mapped, and so on. The direct-mapped implementation is typically used

(a) v associative-mapped caches(b) k direct-mapped caches**Figure 4.13** Mapping from Main Memory to Cache: k -Way Set Associative

for small degrees of associativity (small values of k) while the associative-mapped implementation is typically used for higher degrees of associativity [JACO08].

For set-associative mapping, the cache control logic interprets a memory address as three fields: Tag, Set, and Word. The d set bits specify one of $v = 2^d$ sets. The s bits of the Tag and Set fields specify one of the 2^s blocks of main memory. Figure 4.14 illustrates the cache control logic. With fully associative mapping, the tag in a memory address is quite large and must be compared to the tag of every line in the cache. With k -way set-associative mapping, the tag in a memory address is much smaller and is only compared to the k tags within a single set. To summarize,

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes

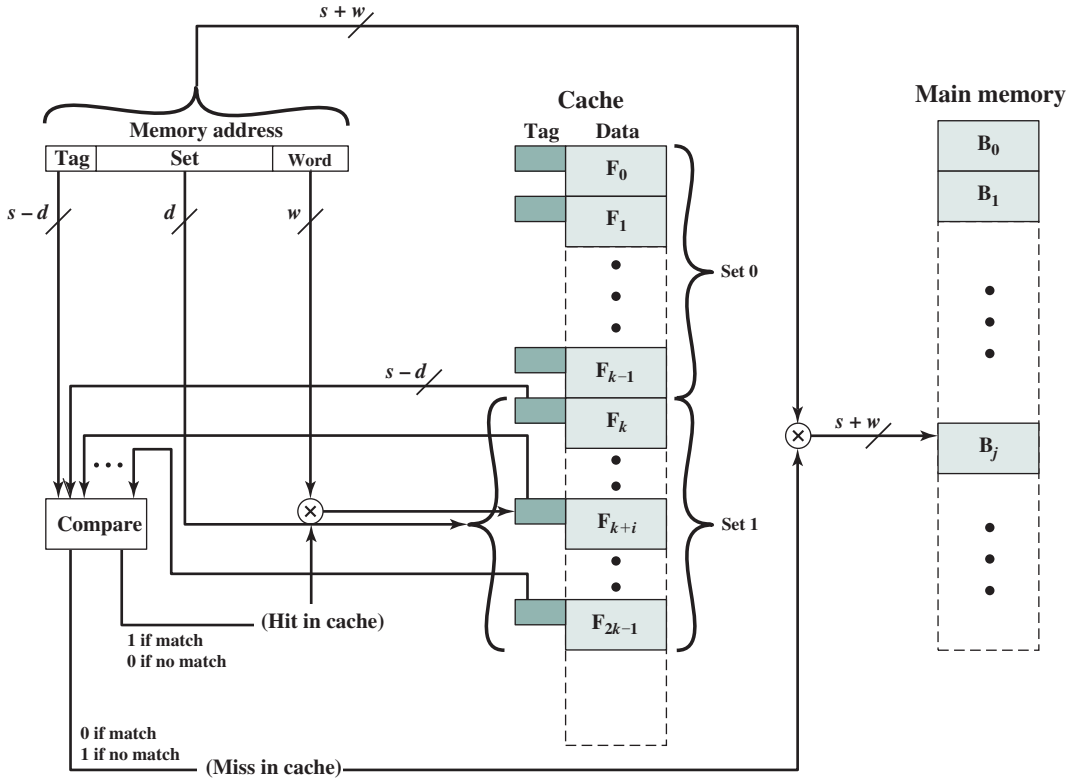


Figure 4.14 k -Way Set-Associative Cache Organization

- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $\frac{2^{s+w}}{2^w} = 2^s$
- Number of lines in set = k
- Number of sets = $v = 2^d$
- Number of lines in cache = $m = kv = k \times 2^d$
- Size of cache = $k \times 2^{d+w}$ words or bytes
- Size of tag = $(s-d)$ bits

EXAMPLE 4.2c Figure 4.15 shows our example using set-associative mapping with two lines in each set, referred to as two-way set-associative. The 13-bit set number identifies a unique set of two lines within the cache. It also gives the number of the block in main memory, modulo 2^{13} . This determines the mapping of blocks into lines. Thus, blocks 000000, 008000, ..., FF8000 of main memory map into cache set 0. Any of those blocks can be loaded into either of the two lines in the set. Note that no two blocks that map into the same cache set have the same tag number. For a read operation, the 13-bit set number is used to determine which set of two lines is to be examined. Both lines in the set are examined for a match with the tag number of the address to be accessed.

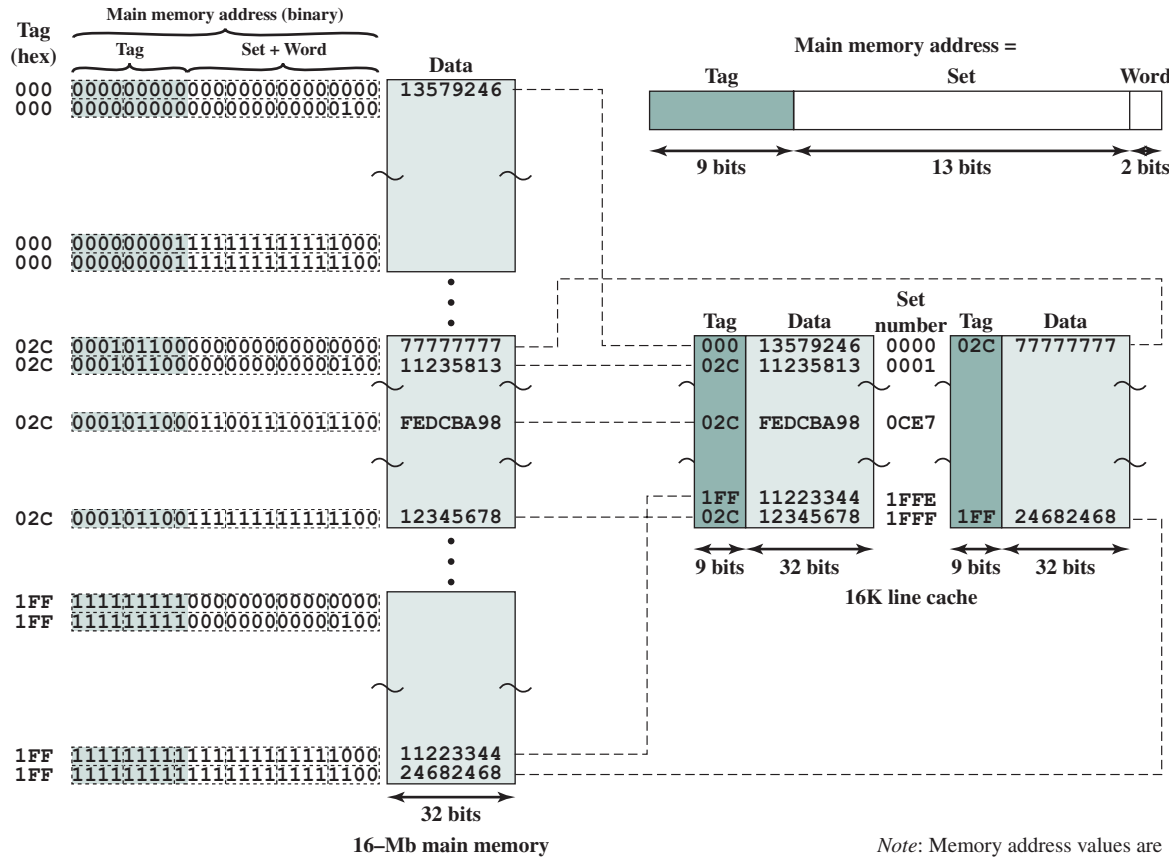


Figure 4.15 Two-Way Set-Associative Mapping Example

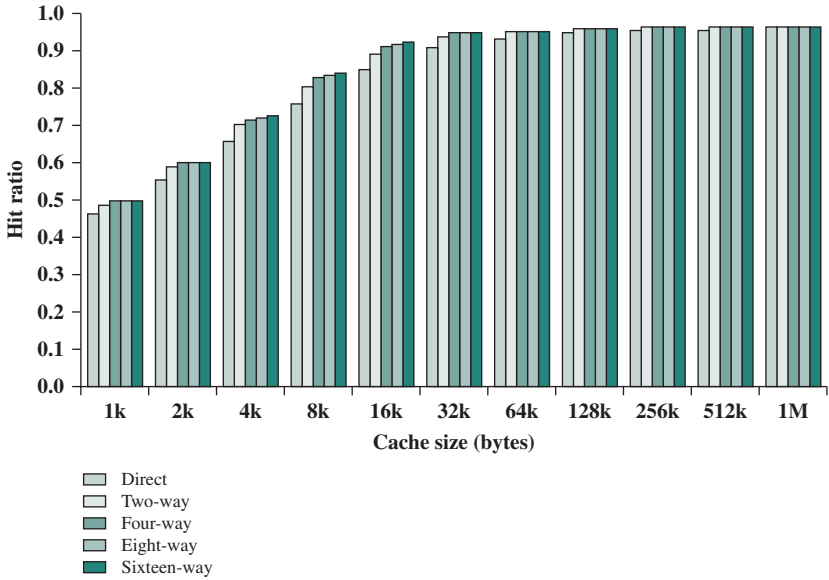


Figure 4.16 Varying Associativity over Cache Size

In the extreme case of $v = m, k = 1$, the set-associative technique reduces to direct mapping, and for $v = 1, k = m$, it reduces to associative mapping. The use of two lines per set ($v = m/2, k = 2$) is the most common set-associative organization. It significantly improves the hit ratio over direct mapping. Four-way set associative ($v = m/4, k = 4$) makes a modest additional improvement for a relatively small additional cost [MAYB84, HILL89]. Further increases in the number of lines per set have little effect.

Figure 4.16 shows the results of one simulation study of set-associative cache performance as a function of cache size [GENU04]. The difference in performance between direct and two-way set associative is significant up to at least a cache size of 64 kB. Note also that the difference between two-way and four-way at 4 kB is much less than the difference in going from for 4 kB to 8 kB in cache size. The complexity of the cache increases in proportion to the associativity, and in this case would not be justifiable against increasing cache size to 8 or even 16 kB. A final point to note is that beyond about 32 kB, increase in cache size brings no significant increase in performance.

The results of Figure 4.16 are based on simulating the execution of a GCC compiler. Different applications may yield different results. For example, [CANT01] reports on the results for cache performance using many of the CPU2000 SPEC benchmarks. The results of [CANT01] in comparing hit ratio to cache size follow the same pattern as Figure 4.16, but the specific values are somewhat different.



Replacement Algorithms

Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced. For direct mapping, there is only one possible line for any particular block, and no choice is possible. For the associative and set-associative techniques, a replacement algorithm is needed. To achieve high speed, such an algorithm must be implemented in hardware. A number of algorithms have been tried. We mention four of the most common. Probably the most effective is **least recently used (LRU)**: Replace that block in the set that has been in the cache longest with no reference to it. For two-way set associative, this is easily implemented. Each line includes a USE bit. When a line is referenced, its USE bit is set to 1 and the USE bit of the other line in that set is set to 0. When a block is to be read into the set, the line whose USE bit is 0 is used. Because we are assuming that more recently used memory locations are more likely to be referenced, LRU should give the best hit ratio. LRU is also relatively easy to implement for a fully associative cache. The cache mechanism maintains a separate list of indexes to all the lines in the cache. When a line is referenced, it moves to the front of the list. For replacement, the line at the back of the list is used. Because of its simplicity of implementation, LRU is the most popular replacement algorithm.

Another possibility is first-in-first-out (FIFO): Replace that block in the set that has been in the cache longest. FIFO is easily implemented as a round-robin or circular buffer technique. Still another possibility is least frequently used (LFU): Replace that block in the set that has experienced the fewest references. LFU could be implemented by associating a counter with each line. A technique not based on usage (i.e., not LRU, LFU, FIFO, or some variant) is to pick a line at random from among the candidate lines. Simulation studies have shown that random replacement provides only slightly inferior performance to an algorithm based on usage [SMIT82].

Write Policy

When a block that is resident in the cache is to be replaced, there are two cases to consider. If the old block in the cache has not been altered, then it may be overwritten with a new block without first writing out the old block. If at least one write operation has been performed on a word in that line of the cache, then main memory must be updated by writing the line of cache out to the block of memory before bringing in the new block. A variety of write policies, with performance and economic trade-offs, is possible. There are two problems to contend with. First, more than one device may have access to main memory. For example, an I/O module may be able to read-write directly to memory. If a word has been altered only in the cache, then the corresponding memory word is invalid. Further, if the I/O device has altered main memory, then the cache word is invalid. A more complex problem occurs when multiple processors are attached to the same bus and each processor has its own local cache. Then, if a word is altered in one cache, it could conceivably invalidate a word in other caches.

The simplest technique is called **write through**. Using this technique, all write operations are made to main memory as well as to the cache, ensuring that main memory is always valid. Any other processor-cache module can monitor traffic to main memory to maintain consistency within its own cache. The main disadvantage

of this technique is that it generates substantial memory traffic and may create a bottleneck. An alternative technique, known as **write back**, minimizes memory writes. With write back, updates are made only in the cache. When an update occurs, a **dirty bit**, or **use bit**, associated with the line is set. Then, when a block is replaced, it is written back to main memory if and only if the dirty bit is set. The problem with write back is that portions of main memory are invalid, and hence accesses by I/O modules can be allowed only through the cache. This makes for complex circuitry and a potential bottleneck. Experience has shown that the percentage of memory references that are writes is on the order of 15% [SMIT82]. However, for HPC applications, this number may approach 33% (vector-vector multiplication) and can go as high as 50% (matrix transposition).

EXAMPLE 4.3 Consider a cache with a line size of 32 bytes and a main memory that requires 30 ns to transfer a 4-byte word. For any line that is written at least once before being swapped out of the cache, what is the average number of times that the line must be written before being swapped out for a write-back cache to be more efficient than a write-through cache?

For the write-back case, each dirty line is written back once, at swap-out time, taking $8 \times 30 = 240$ ns. For the write-through case, each update of the line requires that one word be written out to main memory, taking 30 ns. Therefore, if the average line that gets written at least once gets written more than 8 times before swap out, then write back is more efficient.

In a bus organization in which more than one device (typically a processor) has a cache and main memory is shared, a new problem is introduced. If data in one cache are altered, this invalidates not only the corresponding word in main memory, but also that same word in other caches (if any other cache happens to have that same word). Even if a write-through policy is used, the other caches may contain invalid data. A system that prevents this problem is said to maintain cache coherency. Possible approaches to cache coherency include the following:

- **Bus watching with write through:** Each cache controller monitors the address lines to detect write operations to memory by other bus masters. If another master writes to a location in shared memory that also resides in the cache memory, the cache controller invalidates that cache entry. This strategy depends on the use of a write-through policy by all cache controllers.
- **Hardware transparency:** Additional hardware is used to ensure that all updates to main memory via cache are reflected in all caches. Thus, if one processor modifies a word in its cache, this update is written to main memory. In addition, any matching words in other caches are similarly updated.
- **Noncacheable memory:** Only a portion of main memory is shared by more than one processor, and this is designated as noncacheable. In such a system, all accesses to shared memory are cache misses, because the shared memory is never copied into the cache. The noncacheable memory can be identified using chip-select logic or high-address bits.

Cache coherency is an active field of research. This topic is explored further in Part Five.

Line Size

Another design element is the line size. When a block of data is retrieved and placed in the cache, not only the desired word but also some number of adjacent words are retrieved. As the block size increases from very small to larger sizes, the hit ratio will at first increase because of the principle of locality, which states that data in the vicinity of a referenced word are likely to be referenced in the near future. As the block size increases, more useful data are brought into the cache. The hit ratio will begin to decrease, however, as the block becomes even bigger and the probability of using the newly fetched information becomes less than the probability of reusing the information that has to be replaced. Two specific effects come into play:

- Larger blocks reduce the number of blocks that fit into a cache. Because each block fetch overwrites older cache contents, a small number of blocks results in data being overwritten shortly after they are fetched.
- As a block becomes larger, each additional word is farther from the requested word and therefore less likely to be needed in the near future.

The relationship between block size and hit ratio is complex, depending on the locality characteristics of a particular program, and no definitive optimum value has been found. A size of from 8 to 64 bytes seems reasonably close to optimum [SMIT87, PRZY88, PRZY90, HAND98]. For HPC systems, 64- and 128-byte cache line sizes are most frequently used.

Number of Caches

When caches were originally introduced, the typical system had a single cache. More recently, the use of multiple caches has become the norm. Two aspects of this design issue concern the number of levels of caches and the use of unified versus split caches.

MULTILEVEL CACHES As logic density has increased, it has become possible to have a cache on the same chip as the processor: the on-chip cache. Compared with a cache reachable via an external bus, the on-chip cache reduces the processor's external bus activity and therefore speeds up execution times and increases overall system performance. When the requested instruction or data is found in the on-chip cache, the bus access is eliminated. Because of the short data paths internal to the processor, compared with bus lengths, on-chip cache accesses will complete appreciably faster than would even zero-wait state bus cycles. Furthermore, during this period the bus is free to support other transfers.

The inclusion of an on-chip cache leaves open the question of whether an off-chip, or external, cache is still desirable. Typically, the answer is yes, and most contemporary designs include both on-chip and external caches. The simplest such organization is known as a two-level cache, with the internal level 1 (L1) and the external cache designated as level 2 (L2). The reason for including an L2 cache is the following: If there is no L2 cache and the processor makes an access request for a memory location not in the L1 cache, then the processor must access DRAM or

ROM memory across the bus. Due to the typically slow bus speed and slow memory access time, this results in poor performance. On the other hand, if an L2 SRAM (static RAM) cache is used, then frequently the missing information can be quickly retrieved. If the SRAM is fast enough to match the bus speed, then the data can be accessed using a zero-wait state transaction, the fastest type of bus transfer.

Two features of contemporary cache design for multilevel caches are noteworthy. First, for an off-chip L2 cache, many designs do not use the system bus as the path for transfer between the L2 cache and the processor, but use a separate data path, so as to reduce the burden on the system bus. Second, with the continued shrinkage of processor components, a number of processors now incorporate the L2 cache on the processor chip, improving performance.

The potential savings due to the use of an L2 cache depends on the hit rates in both the L1 and L2 caches. Several studies have shown that, in general, the use of a second-level cache does improve performance (e.g., see [AZIM92], [NOVI93], [HAND98]). However, the use of multilevel caches does complicate all of the design issues related to caches, including size, replacement algorithm, and write policy; see [HAND98] and [PEIR99] for discussions.

Figure 4.17 shows the results of one simulation study of two-level cache performance as a function of cache size [GENU04]. The figure assumes that both caches have the same line size and shows the total hit ratio. That is, a hit is counted if the desired data appears in either the L1 or the L2 cache. The figure shows the impact of L2 on total hits with respect to L1 size. L2 has little effect on the total number of cache hits until it is at least double the L1 cache size. Note that the steepest part of the slope for an L1 cache of 8 kB is for an L2 cache of 16 kB. Again for an L1 cache of 16 kB, the steepest part of the curve is for an L2 cache size of 32 kB. Prior to that point, the L2 cache has little, if any, impact on total cache performance. The need for the L2 cache to be larger than

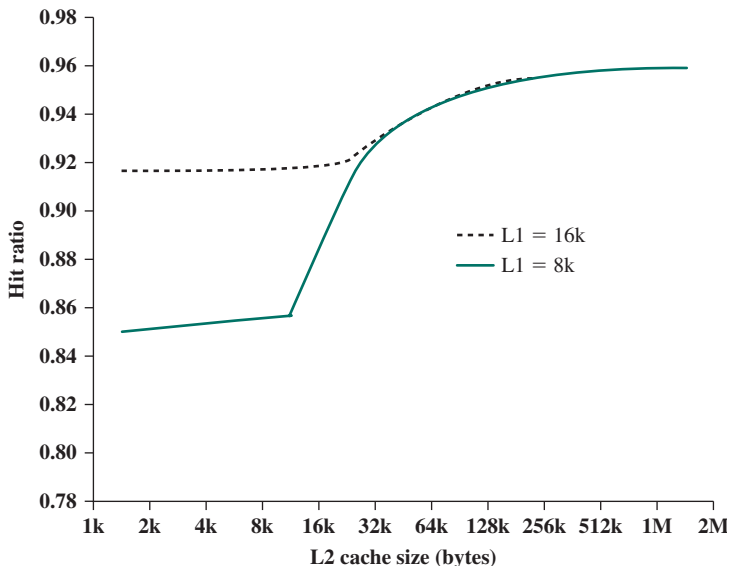


Figure 4.17 Total Hit Ratio (L1 and L2) for 8-kB and 16-kB L1

the L1 cache to affect performance makes sense. If the L2 cache has the same line size and capacity as the L1 cache, its contents will more or less mirror those of the L1 cache.

With the increasing availability of on-chip area available for cache, most contemporary microprocessors have moved the L2 cache onto the processor chip and added an L3 cache. Originally, the L3 cache was accessible over the external bus. More recently, most microprocessors have incorporated an on-chip L3 cache. In either case, there appears to be a performance advantage to adding the third level (e.g., see [GHAI98]). Further, large systems, such as the IBM mainframe zEnterprise systems, now incorporate 3 on-chip cache levels and a fourth level of cache shared across multiple chips [CURR11].

UNIFIED VERSUS SPLIT CACHES When the on-chip cache first made an appearance, many of the designs consisted of a single cache used to store references to both data and instructions. More recently, it has become common to split the cache into two: one dedicated to instructions and one dedicated to data. These two caches both exist at the same level, typically as two L1 caches. When the processor attempts to fetch an instruction from main memory, it first consults the instruction L1 cache, and when the processor attempts to fetch data from main memory, it first consults the data L1 cache.

There are two potential advantages of a unified cache:

- For a given cache size, a unified cache has a higher hit rate than split caches because it balances the load between instruction and data fetches automatically. That is, if an execution pattern involves many more instruction fetches than data fetches, then the cache will tend to fill up with instructions, and if an execution pattern involves relatively more data fetches, the opposite will occur.
- Only one cache needs to be designed and implemented.

The trend is toward split caches at the L1 and unified caches for higher levels, particularly for superscalar machines, which emphasize parallel instruction execution and the prefetching of predicted future instructions. The key advantage of the split cache design is that it eliminates contention for the cache between the instruction fetch/decode unit and the execution unit. This is important in any design that relies on the pipelining of instructions. Typically, the processor will fetch instructions ahead of time and fill a buffer, or pipeline, with instructions to be executed. Suppose now that we have a unified instruction/data cache. When the execution unit performs a memory access to load and store data, the request is submitted to the unified cache. If, at the same time, the instruction prefetcher issues a read request to the cache for an instruction, that request will be temporarily blocked so that the cache can service the execution unit first, enabling it to complete the currently executing instruction. This cache contention can degrade performance by interfering with efficient use of the instruction pipeline. The split cache structure overcomes this difficulty.

4.4 PENTIUM 4 CACHE ORGANIZATION

The evolution of cache organization is seen clearly in the evolution of Intel microprocessors (Table 4.4). The 80386 does not include an on-chip cache. The 80486 includes a single on-chip cache of 8 kB, using a line size of 16 bytes and a four-way

Table 4.4 Intel Cache Evolution

Problem	Solution	Processor on Which Feature First Appears
External memory slower than the system bus.	Add external cache using faster memory technology.	386
Increased processor speed results in external bus becoming a bottleneck for cache access.	Move external cache on-chip, operating at the same speed as the processor.	486
Internal cache is rather small, due to limited space on chip.	Add external L2 cache using faster technology than main memory.	486
Contention occurs when both the Instruction Prefetcher and the Execution Unit simultaneously require access to the cache. In that case, the Prefetcher is stalled while the Execution Unit's data access takes place.	Create separate data and instruction caches.	Pentium
Increased processor speed results in external bus becoming a bottleneck for L2 cache access.	Create separate back-side bus that runs at higher speed than the main (front-side) external bus. The BSB is dedicated to the L2 cache.	Pentium Pro
	Move L2 cache on to the processor chip.	Pentium II
Some applications deal with massive databases and must have rapid access to large amounts of data. The on-chip caches are too small.	Add external L3 cache.	Pentium III
	Move L3 cache on-chip.	Pentium 4

set-associative organization. All of the Pentium processors include two on-chip L1 caches, one for data and one for instructions. For the Pentium 4, the L1 data cache is 16 kB, using a line size of 64 bytes and a four-way set-associative organization. The Pentium 4 instruction cache is described subsequently. The Pentium II also includes an L2 cache that feeds both of the L1 caches. The L2 cache is eight-way set associative with a size of 512 kB and a line size of 128 bytes. An L3 cache was added for the Pentium III and became on-chip with high-end versions of the Pentium 4.

Figure 4.18 provides a simplified view of the Pentium 4 organization, highlighting the placement of the three caches. The processor core consists of four major components:

- **Fetch/decode unit:** Fetches program instructions in order from the L2 cache, decodes these into a series of micro-operations, and stores the results in the L1 instruction cache.
- **Out-of-order execution logic:** Schedules execution of the micro-operations subject to data dependencies and resource availability; thus, micro-operations may be scheduled for execution in a different order than they were fetched from the instruction stream. As time permits, this unit schedules speculative execution of micro-operations that may be required in the future.

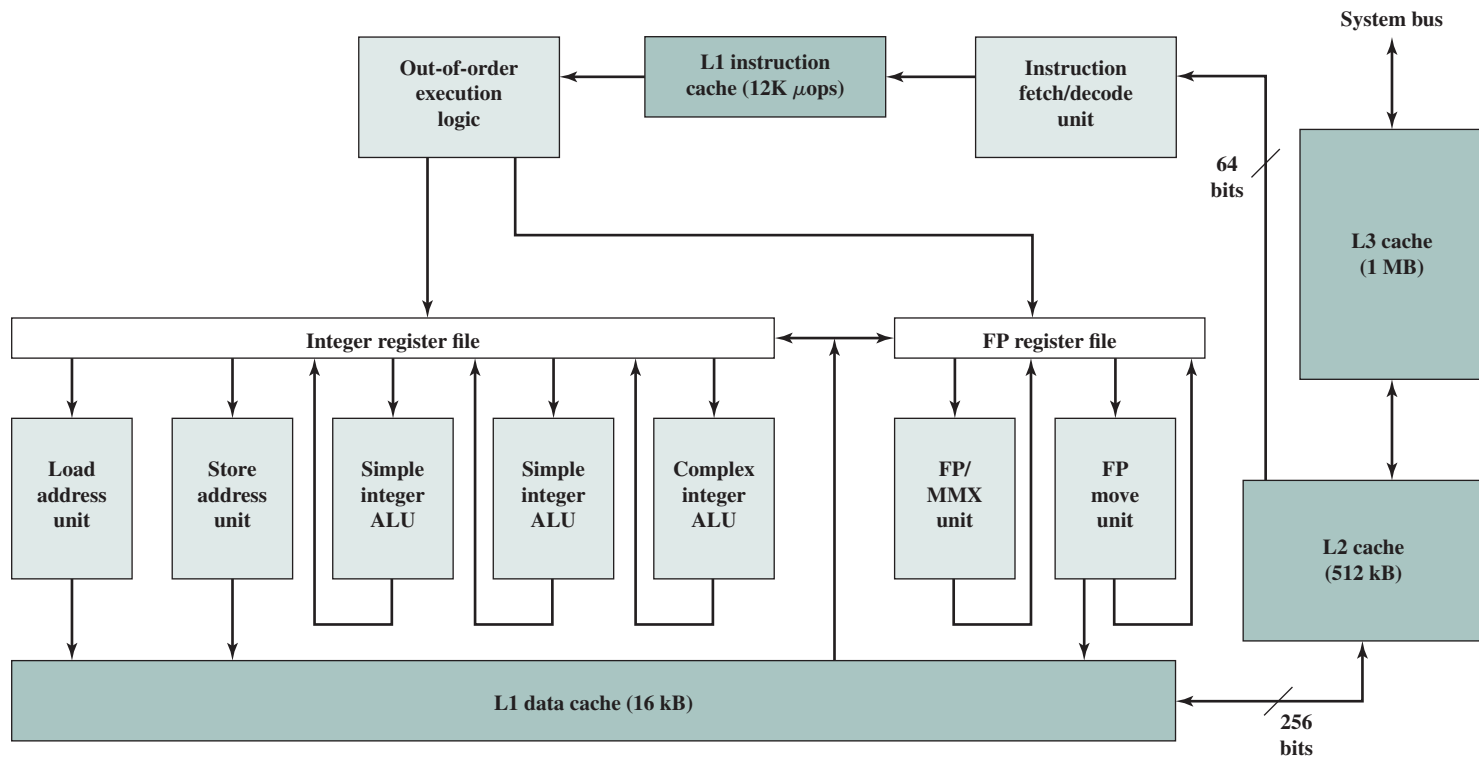


Figure 4.18 Pentium 4 Block Diagram

Table 4.5 Pentium 4 Cache Operating Modes

Control Bits		Operating Mode		
CD	NW	Cache Fills	Write Throughs	Invalidates
0	0	Enabled	Enabled	Enabled
1	0	Disabled	Enabled	Enabled
1	1	Disabled	Disabled	Disabled

Note: CD = 0; NW = 1 is an invalid combination.

- **Execution units:** These units execute micro-operations, fetching the required data from the L1 data cache and temporarily storing results in registers.
- **Memory subsystem:** This unit includes the L2 and L3 caches and the system bus, which is used to access main memory when the L1 and L2 caches have a cache miss and to access the system I/O resources.

Unlike the organization used in all previous Pentium models, and in most other processors, the Pentium 4 instruction cache sits between the instruction decode logic and the execution core. The reasoning behind this design decision is as follows: As discussed more fully in Chapter 16, the Pentium process decodes, or translates, Pentium machine instructions into simple RISC-like instructions called micro-operations. The use of simple, fixed-length micro-operations enables the use of superscalar pipelining and scheduling techniques that enhance performance. However, the Pentium machine instructions are cumbersome to decode; they have a variable number of bytes and many different options. It turns out that performance is enhanced if this decoding is done independently of the scheduling and pipelining logic. We return to this topic in Chapter 16.

The data cache employs a write-back policy: Data are written to main memory only when they are removed from the cache and there has been an update. The Pentium 4 processor can be dynamically configured to support write-through caching.

The L1 data cache is controlled by two bits in one of the control registers, labeled the CD (cache disable) and NW (not write-through) bits (Table 4.5). There are also two Pentium 4 instructions that can be used to control the data cache: INVD invalidates (flushes) the internal cache memory and signals the external cache (if any) to invalidate. WBINVD writes back and invalidates internal cache and then writes back and invalidates external cache.

Both the L2 and L3 caches are eight-way set-associative with a line size of 128 bytes.

4.5 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

access time associative mapping cache hit	cache line cache memory cache miss	cache set data cache direct access
---	--	--

direct mapping high-performance computing (HPC) hit hit ratio instruction cache L1 cache L2 cache L3 cache line locality	logical cache memory hierarchy miss multilevel cache physical address physical cache random access replacement algorithm secondary memory sequential access set-associative mapping	spatial locality split cache tag temporal locality unified cache virtual address virtual cache write back write through
---	---	---

Review Questions

- 4.1 What are the differences among sequential access, direct access, and random access?
- 4.2 What is the general relationship among access time, memory cost, and capacity?
- 4.3 How does the principle of locality relate to the use of multiple memory levels?
- 4.4 What are the differences among direct mapping, associative mapping, and set-associative mapping?
- 4.5 For a direct-mapped cache, a main memory address is viewed as consisting of three fields. List and define the three fields.
- 4.6 For an associative cache, a main memory address is viewed as consisting of two fields. List and define the two fields.
- 4.7 For a set-associative cache, a main memory address is viewed as consisting of three fields. List and define the three fields.
- 4.8 What is the distinction between spatial locality and temporal locality?
- 4.9 In general, what are the strategies for exploiting spatial locality and temporal locality?

Problems

- 4.1 A set-associative cache consists of 64 lines, or slots, divided into four-line sets. Main memory contains 4K blocks of 128 words each. Show the format of main memory addresses.
- 4.2 A two-way set-associative cache has lines of 16 bytes and a total size of 8 kB. The 64-MB main memory is byte addressable. Show the format of main memory addresses.
- 4.3 For the hexadecimal main memory addresses 111111, 666666, BBBB, show the following information, in hexadecimal format:
 - a. Tag, Line, and Word values for a direct-mapped cache, using the format of Figure 4.10
 - b. Tag and Word values for an associative cache, using the format of Figure 4.12
 - c. Tag, Set, and Word values for a two-way set-associative cache, using the format of Figure 4.15
- 4.4 List the following values:
 - a. For the direct cache example of Figure 4.10: address length, number of addressable units, block size, number of blocks in main memory, number of lines in cache, size of tag
 - b. For the associative cache example of Figure 4.12: address length, number of addressable units, block size, number of blocks in main memory, number of lines in cache, size of tag

- c. For the two-way set-associative cache example of Figure 4.15: address length, number of addressable units, block size, number of blocks in main memory, number of lines in set, number of sets, number of lines in cache, size of tag
- 4.5 Consider a 32-bit microprocessor that has an on-chip 16-kB four-way set-associative cache. Assume that the cache has a line size of four 32-bit words. Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss. Where in the cache is the word from memory location ABCDE8F8 mapped?
- 4.6 Given the following specifications for an external cache memory: four-way set associative; line size of two 16-bit words; able to accommodate a total of 4K 32-bit words from main memory; used with a 16-bit processor that issues 24-bit addresses. Design the cache structure with all pertinent information and show how it interprets the processor's addresses.
- 4.7 The Intel 80486 has an on-chip, unified cache. It contains 8 kB and has a four-way set-associative organization and a block length of four 32-bit words. The cache is organized into 128 sets. There is a single "line valid bit" and three bits, B0, B1, and B2 (the "LRU" bits), per line. On a cache miss, the 80486 reads a 16-byte line from main memory in a bus memory read burst. Draw a simplified diagram of the cache and show how the different fields of the address are interpreted.
- 4.8 Consider a machine with a byte addressable main memory of 2^{16} bytes and block size of 8 bytes. Assume that a direct mapped cache consisting of 32 lines is used with this machine.
- How is a 16-bit memory address divided into tag, line number, and byte number?
 - Into what line would bytes with each of the following addresses be stored?
- | | | | |
|------|------|------|------|
| 0001 | 0001 | 0001 | 1011 |
| 1100 | 0011 | 0011 | 0100 |
| 1101 | 0000 | 0001 | 1101 |
| 1010 | 1010 | 1010 | 1010 |
- Suppose the byte with address 0001 1010 0001 1010 is stored in the cache. What are the addresses of the other bytes stored along with it?
 - How many total bytes of memory can be stored in the cache?
 - Why is the tag also stored in the cache?
- 4.9 For its on-chip cache, the Intel 80486 uses a replacement algorithm referred to as **pseudo least recently used**. Associated with each of the 128 sets of four lines (labeled L0, L1, L2, L3) are three bits B0, B1, and B2. The replacement algorithm works as follows: When a line must be replaced, the cache will first determine whether the most recent use was from L0 and L1 or L2 and L3. Then the cache will determine which of the pair of blocks was least recently used and mark it for replacement. Figure 4.19 illustrates the logic.
- Specify how the bits B0, B1, and B2 are set and then describe in words how they are used in the replacement algorithm depicted in Figure 4.19.
 - Show that the 80486 algorithm approximates a true LRU algorithm. *Hint:* Consider the case in which the most recent order of usage is L0, L2, L3, L1.
 - Demonstrate that a true LRU algorithm would require 6 bits per set.
- 4.10 A set-associative cache has a block size of four 16-bit words and a set size of 2. The cache can accommodate a total of 4096 words. The main memory size that is cacheable is $64K \times 32$ bits. Design the cache structure and show how the processor's addresses are interpreted.
- 4.11 Consider a memory system that uses a 32-bit address to address at the byte level, plus a cache that uses a 64-byte line size.
- Assume a direct mapped cache with a tag field in the address of 20 bits. Show the address format and determine the following parameters: number of addressable units, number of blocks in main memory, number of lines in cache, size of tag.

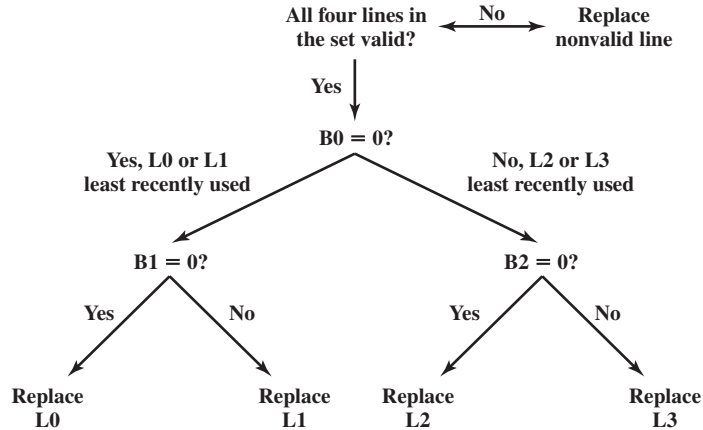


Figure 4.19 Intel 80486 On-Chip Cache Replacement Strategy

- b. Assume an associative cache. Show the address format and determine the following parameters: number of addressable units, number of blocks in main memory, number of lines in cache, size of tag.
 - c. Assume a four-way set-associative cache with a tag field in the address of 9 bits. Show the address format and determine the following parameters: number of addressable units, number of blocks in main memory, number of lines in set, number of sets in cache, number of lines in cache, size of tag.
- 4.12** Consider a computer with the following characteristics: total of 1 MB of main memory; word size of 1 byte; block size of 16 bytes; and cache size of 64 kB.
- a. For the main memory addresses of F0010, 01234, and CABBE, give the corresponding tag, cache line address, and word offsets for a direct-mapped cache.
 - b. Give any two main memory addresses with different tags that map to the same cache slot for a direct-mapped cache.
 - c. For the main memory addresses of F0010 and CABBE, give the corresponding tag and offset values for a fully-associative cache.
 - d. For the main memory addresses of F0010 and CABBE, give the corresponding tag, cache set, and offset values for a two-way set-associative cache.
- 4.13** Describe a simple technique for implementing an LRU replacement algorithm in a four-way set-associative cache.
- 4.14** Consider again Example 4.3. How does the answer change if the main memory uses a block transfer capability that has a first-word access time of 30 ns and an access time of 5 ns for each word thereafter?
- 4.15** Consider the following code:
- ```

for (i = 0; i < 20; i++)
 for (j = 0; j < 10; j++)
 a[i] = a[i]*j

```
- a. Give one example of the spatial locality in the code.
  - b. Give one example of the temporal locality in the code.
- 4.16** Generalize Equations (4.2) and (4.3), in Appendix 4A, to  $N$ -level memory hierarchies.
- 4.17** A computer system contains a main memory of 32K 16-bit words. It also has a 4K word cache divided into four-line sets with 64 words per line. Assume that the cache is initially empty. The processor fetches words from locations 0, 1, 2, ..., 4351 in that

order. It then repeats this fetch sequence nine more times. The cache is 10 times faster than main memory. Estimate the improvement resulting from the use of the cache. Assume an LRU policy for block replacement.

- 4.18** Consider a cache of 4 lines of 16 bytes each. Main memory is divided into blocks of 16 bytes each. That is, block 0 has bytes with addresses 0 through 15, and so on. Now consider a program that accesses memory in the following sequence of addresses: Once: 63 through 70.  
Loop ten times: 15 through 32; 80 through 95.
- Suppose the cache is organized as direct mapped. Memory blocks 0, 4, and so on are assigned to line 1; blocks 1, 5, and so on to line 2; and so on. Compute the hit ratio.
  - Suppose the cache is organized as two-way set associative, with two sets of two lines each. Even-numbered blocks are assigned to set 0 and odd-numbered blocks are assigned to set 1. Compute the hit ratio for the two-way set-associative cache using the least recently used replacement scheme.
- 4.19** Consider a memory system with the following parameters:

$$T_c = 100 \text{ ns} \quad C_c = 10^{-4} \text{ \$/bit}$$

$$T_m = 1200 \text{ ns} \quad C_m = 10^{-5} \text{ \$/bit}$$

- What is the cost of 1 MB of main memory?
  - What is the cost of 1 MB of main memory using cache memory technology?
  - If the effective access time is 10% greater than the cache access time, what is the hit ratio  $H$ ?
- 4.20**
- Consider an L1 cache with an access time of 1 ns and a hit ratio of  $H = 0.95$ . Suppose that we can change the cache design (size of cache, cache organization) such that we increase  $H$  to 0.97, but increase access time to 1.5 ns. What conditions must be met for this change to result in improved performance?
  - Explain why this result makes intuitive sense.
- 4.21** Consider a single-level cache with an access time of 2.5 ns, a line size of 64 bytes, and a hit ratio of  $H = 0.95$ . Main memory uses a block transfer capability that has a first-word (4 bytes) access time of 50 ns and an access time of 5 ns for each word thereafter.
- What is the access time when there is a cache miss? Assume that the cache waits until the line has been fetched from main memory and then re-executes for a hit.
  - Suppose that increasing the line size to 128 bytes increases the  $H$  to 0.97. Does this reduce the average memory access time?
- 4.22** A computer has a cache, main memory, and a disk used for virtual memory. If a referenced word is in the cache, 20 ns are required to access it. If it is in main memory but not in the cache, 60 ns are needed to load it into the cache, and then the reference is started again. If the word is not in main memory, 12 ms are required to fetch the word from disk, followed by 60 ns to copy it to the cache, and then the reference is started again. The cache hit ratio is 0.9 and the main memory hit ratio is 0.6. What is the average time in nanoseconds required to access a referenced word on this system?
- 4.23** Consider a cache with a line size of 64 bytes. Assume that on average 30% of the lines in the cache are dirty. A word consists of 8 bytes.
- Assume there is a 3% miss rate (0.97 hit ratio). Compute the amount of main memory traffic, in terms of bytes per instruction for both write-through and write-back policies. Memory is read into cache one line at a time. However, for write back, a single word can be written from cache to main memory.
  - Repeat part a for a 5% rate.
  - Repeat part a for a 7% rate.
  - What conclusion can you draw from these results?
- 4.24** On the Motorola 68020 microprocessor, a cache access takes two clock cycles. Data access from main memory over the bus to the processor takes three clock cycles in the

case of no wait state insertion; the data are delivered to the processor in parallel with delivery to the cache.

- a. Calculate the effective length of a memory cycle given a hit ratio of 0.9 and a clocking rate of 16.67 MHz.
  - b. Repeat the calculations assuming insertion of two wait states of one cycle each per memory cycle. What conclusion can you draw from the results?
- 4.25** Assume a processor having a memory cycle time of 300 ns and an instruction processing rate of 1 MIPS. On average, each instruction requires one bus memory cycle for instruction fetch and one for the operand it involves.
- a. Calculate the utilization of the bus by the processor.
  - b. Suppose the processor is equipped with an instruction cache and the associated hit ratio is 0.5. Determine the impact on bus utilization.
- 4.26** The performance of a single-level cache system for a read operation can be characterized by the following equation:

$$T_a = T_c + (1 - H)T_m$$

where  $T_a$  is the average access time,  $T_c$  is the cache access time,  $T_m$  is the memory access time (memory to processor register), and  $H$  is the hit ratio. For simplicity, we assume that the word in question is loaded into the cache in parallel with the load to processor register. This is the same form as Equation (4.2).

- a. Define  $T_b$  = time to transfer a line between cache and main memory, and  $W$  = fraction of write references. Revise the preceding equation to account for writes as well as reads, using a write-through policy.
  - b. Define  $W_b$  as the probability that a line in the cache has been altered. Provide an equation for  $T_a$  for the write-back policy.
- 4.27** For a system with two levels of cache, define  $T_{c_1}$  = first – level cache access time;  $T_{c_2}$  = second – level cache access time;  $T_m$  = memory access time;  $H_1$  = first – level cache hit ratio;  $H_2$  = combined first/second level cache hit ratio. Provide an equation for  $T_a$  for a read operation.
- 4.28** Assume the following performance characteristics on a cache read miss: one clock cycle to send an address to main memory and four clock cycles to access a 32-bit word from main memory and transfer it to the processor and cache.
- a. If the cache line size is one word, what is the miss penalty (i.e., additional time required for a read in the event of a read miss)?
  - b. What is the miss penalty if the cache line size is four words and a multiple, non-burst transfer is executed?
  - c. What is the miss penalty if the cache line size is four words and a transfer is executed, with one clock cycle per word transfer?
- 4.29** For the cache design of the preceding problem, suppose that increasing the line size from one word to four words results in a decrease of the read miss rate from 3.2% to 1.1%. For both the nonburst transfer and the burst transfer case, what is the average miss penalty, averaged over all reads, for the two different line sizes?

## APPENDIX 4A PERFORMANCE CHARACTERISTICS OF TWO-LEVEL MEMORIES

In this chapter, reference is made to a cache that acts as a buffer between main memory and processor, creating a two-level internal memory. This two-level architecture exploits a property known as locality to provide improved performance over a comparable one-level memory.

The main memory cache mechanism is part of the computer architecture, implemented in hardware and typically invisible to the operating system. There are two other instances of a two-level memory approach that also exploit locality and that are, at least partially, implemented in the operating system: virtual memory and the disk cache (Table 4.6). Virtual memory is explored in Chapter 8; disk cache is beyond the scope of this book but is examined in [STAL15]. In this appendix, we look at some of the performance characteristics of two-level memories that are common to all three approaches.

Locality

The basis for the performance advantage of a two-level memory is a principle known as **locality of reference** [DENN68]. This principle states that memory references tend to cluster. Over a long period of time, the clusters in use change, but over a short period of time, the processor is primarily working with fixed clusters of memory references.

Intuitively, the principle of locality makes sense. Consider the following line of reasoning:

1. Except for branch and call instructions, which constitute only a small fraction of all program instructions, program execution is sequential. Hence, in most cases, the next instruction to be fetched immediately follows the last instruction fetched.
2. It is rare to have a long uninterrupted sequence of procedure calls followed by the corresponding sequence of returns. Rather, a program remains confined to a rather narrow window of procedure-invocation depth. Thus, over a short period of time references to instructions tend to be localized to a few procedures.
3. Most iterative constructs consist of a relatively small number of instructions repeated many times. For the duration of the iteration, computation is therefore confined to a small contiguous portion of a program.
4. In many programs, much of the computation involves processing data structures, such as arrays or sequences of records. In many cases, successive references to these data structures will be to closely located data items.

Table 4.6 Characteristics of Two-Level Memories

|                                     | Main Memory<br>Cache            | Virtual Memory<br>(paging)                  | Disk Cache                                |
|-------------------------------------|---------------------------------|---------------------------------------------|-------------------------------------------|
| Typical access time ratios          | 5:1 (main memory vs. cache)     | 10 <sup>6</sup> :1 (main memory vs. disk)   | 10 <sup>6</sup> :1 (main memory vs. disk) |
| Memory management system            | Implemented by special hardware | Combination of hardware and system software | System software                           |
| Typical block or page size          | 4 to 128 bytes (cache block)    | 64 to 4096 bytes (virtual memory page)      | 64 to 4096 bytes (disk block or pages)    |
| Access of processor to second level | Direct access                   | Indirect access                             | Indirect access                           |



**Table 4.7** Relative Dynamic Frequency of High-Level Language Operations

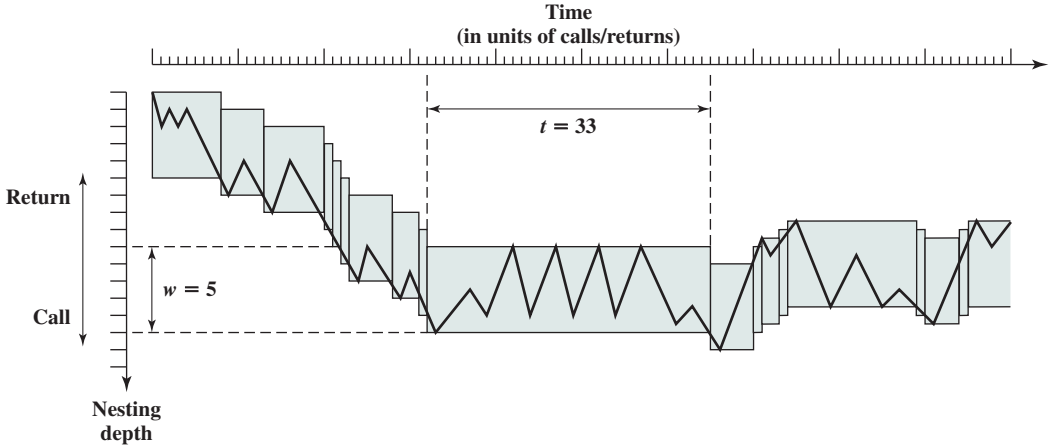
| Study<br>Language<br>Workload | [HUCK83]<br>Pascal<br>Scientific | [KNUT71]<br>FORTRAN<br>Student | [PATT82a]        |             | [TANE78]<br>SAL<br>System |
|-------------------------------|----------------------------------|--------------------------------|------------------|-------------|---------------------------|
|                               |                                  |                                | Pascal<br>System | C<br>System |                           |
| Assign                        | 74                               | 67                             | 45               | 38          | 42                        |
| Loop                          | 4                                | 3                              | 5                | 3           | 4                         |
| Call                          | 1                                | 3                              | 15               | 12          | 12                        |
| IF                            | 20                               | 11                             | 29               | 43          | 36                        |
| GOTO                          | 2                                | 9                              | —                | 3           | —                         |
| Other                         | —                                | 7                              | 6                | 1           | 6                         |

This line of reasoning has been confirmed in many studies. With reference to point 1, a variety of studies have analyzed the behavior of high-level language programs. Table 4.7 includes key results, measuring the appearance of various statement types during execution, from the following studies. The earliest study of programming language behavior, performed by Knuth [KNUT71], examined a collection of FORTRAN programs used as student exercises. Tanenbaum [TANE78] published measurements collected from over 300 procedures used in operating-system programs and written in a language that supports structured programming (SAL). Patterson and Sequein [PATT82a] analyzed a set of measurements taken from compilers and programs for typesetting, computer-aided design (CAD), sorting, and file comparison. The programming languages C and Pascal were studied. Huck [HUCK83] analyzed four programs intended to represent a mix of general-purpose scientific computing, including fast Fourier transform and the integration of systems of differential equations. There is good agreement in the results of this mixture of languages and applications that branching and call instructions represent only a fraction of statements executed during the lifetime of a program. Thus, these studies confirm assertion 1.

With respect to assertion 2, studies reported in [PATT85a] provide confirmation. This is illustrated in Figure 4.20, which shows call-return behavior. Each call is represented by the line moving down and to the right, and each return by the line moving up and to the right. In the figure, a *window* with depth equal to 5 is defined. Only a sequence of calls and returns with a net movement of 6 in either direction causes the window to move. As can be seen, the executing program can remain within a stationary window for long periods of time. A study by the same analysts of C and Pascal programs showed that a window of depth 8 will need to shift only on less than 1% of the calls or returns [TAMI83].

A distinction is made in the literature between spatial locality and temporal locality. **Spatial locality** refers to the tendency of execution to involve a number of memory locations that are clustered. This reflects the tendency of a processor to access instructions sequentially. Spatial location also reflects the tendency of a program to access data locations sequentially, such as when processing a table of data. **Temporal locality** refers to the tendency for a processor to access memory locations that have been used recently. For example, when an iteration loop is executed, the processor executes the same set of instructions repeatedly.





**Figure 4.20** Example Call-Return Behavior of a Program

Traditionally, temporal locality is exploited by keeping recently used instruction and data values in cache memory and by exploiting a cache hierarchy. Spatial locality is generally exploited by using larger cache blocks and by incorporating prefetching mechanisms (fetching items of anticipated use) into the cache control logic. Recently, there has been considerable research on refining these techniques to achieve greater performance, but the basic strategies remain the same.

### Operation of Two-Level Memory

The locality property can be exploited in the formation of a two-level memory. The upper-level memory (M1) is smaller, faster, and more expensive (per bit) than the lower-level memory (M2). M1 is used as a temporary store for part of the contents of the larger M2. When a memory reference is made, an attempt is made to access the item in M1. If this succeeds, then a quick access is made. If not, then a block of memory locations is copied from M2 to M1 and the access then takes place via M1. Because of locality, once a block is brought into M1, there should be a number of accesses to locations in that block, resulting in fast overall service.

To express the average time to access an item, we must consider not only the speeds of the two levels of memory, but also the probability that a given reference can be found in M1. We have

$$\begin{aligned} T_s &= H \times T_1 + (1 - H) \times (T_1 + T_2) \\ &= T_1 + (1 - H) \times T_2 \end{aligned} \quad (4.2)$$

where

$T_s$  = average (system) access time

$T_1$  = access time of M1 (e.g., cache, disk cache)

$T_2$  = access time of M2 (e.g., main memory, disk)

$H$  = hit ratio (fraction of time reference is found in M1)

Figure 4.2 shows average access time as a function of hit ratio. As can be seen, for a high percentage of hits, the average total access time is much closer to that of M1 than M2.

## Performance

Let us look at some of the parameters relevant to an assessment of a two-level memory mechanism. First consider cost. We have

$$C_s = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2} \quad (4.3)$$

where

$C_s$  = average cost per bit for the combined two-level memory

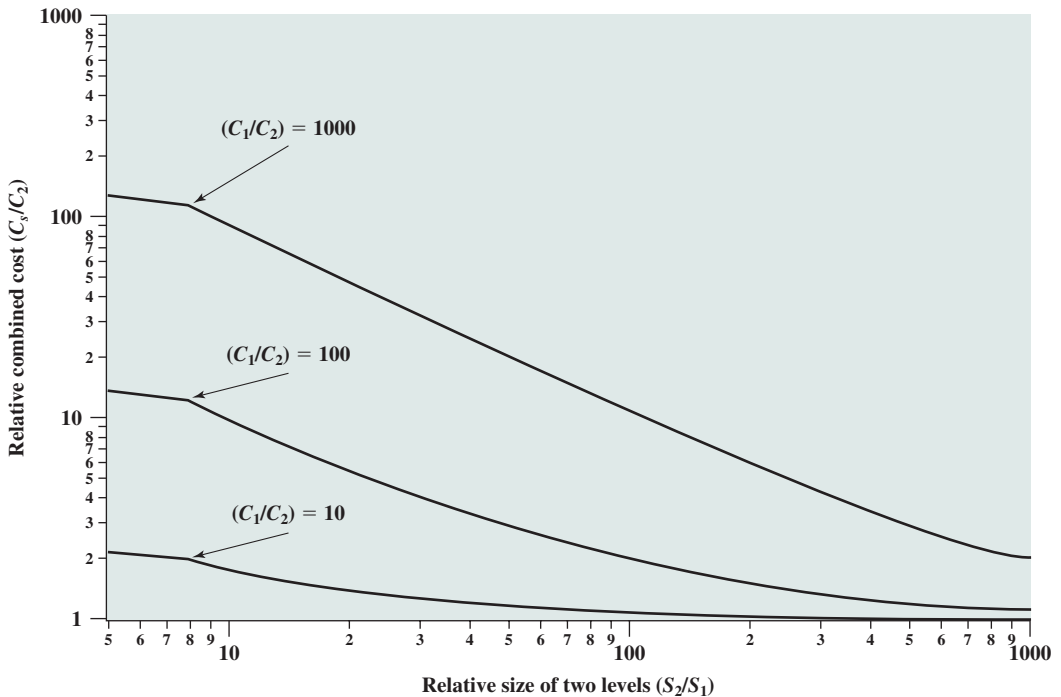
$C_1$  = average cost per bit of upper-level memory M1

$C_2$  = average cost per bit of lower-level memory M2

$S_1$  = size of M1

$S_2$  = size of M2

We would like  $C_s \approx C_2$ . Given that  $C_1 \gg C_2$ , this requires  $S_1 < S_2$ . Figure 4.21 shows the relationship.



**Figure 4.21** Relationship of Average Memory Cost to Relative Memory Size for a Two-Level Memory

Next, consider access time. For a two-level memory to provide a significant performance improvement, we need to have  $T_s$  approximately equal to  $T_1$  ( $T_s \approx T_1$ ). Given that  $T_1$  is much less than  $T_2$  ( $T_1 \ll T_2$ ), a hit ratio of close to 1 is needed.

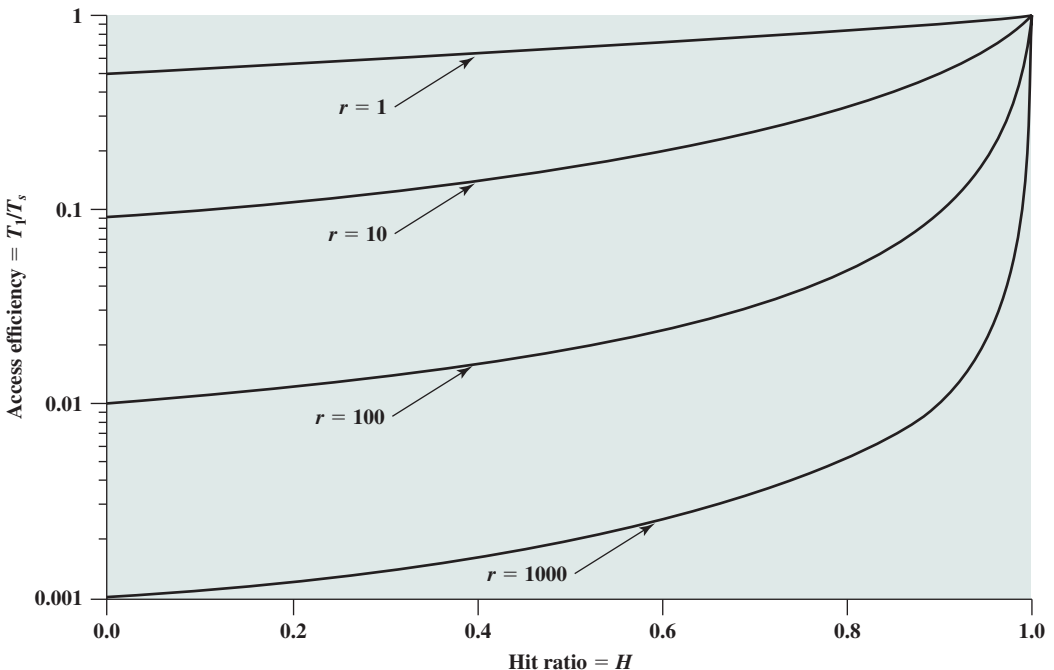
So we would like M1 to be small to hold down cost, and large to improve the hit ratio and therefore the performance. Is there a size of M1 that satisfies both requirements to a reasonable extent? We can answer this question with a series of subquestions:

- What value of hit ratio is needed so that  $T_s \approx T_1$ ?
- What size of M1 will assure the needed hit ratio?
- Does this size satisfy the cost requirement?

To get at this, consider the quantity  $T_1/T_s$ , which is referred to as the *access efficiency*. It is a measure of how close average access time ( $T_s$ ) is to M1 access time ( $T_1$ ). From Equation (4.2),

$$\frac{T_1}{T_s} = \frac{1}{1 + (1 - H) \frac{T_2}{T_1}} \quad (4.4)$$

Figure 4.22 plots  $T_1/T_s$  as a function of the hit ratio  $H$ , with the quantity  $T_2/T_1$  as a parameter. Typically, on-chip cache access time is about 25 to 50 times faster than main memory access time (i.e.,  $T_2/T_1$  is 25 to 50), off-chip cache access time

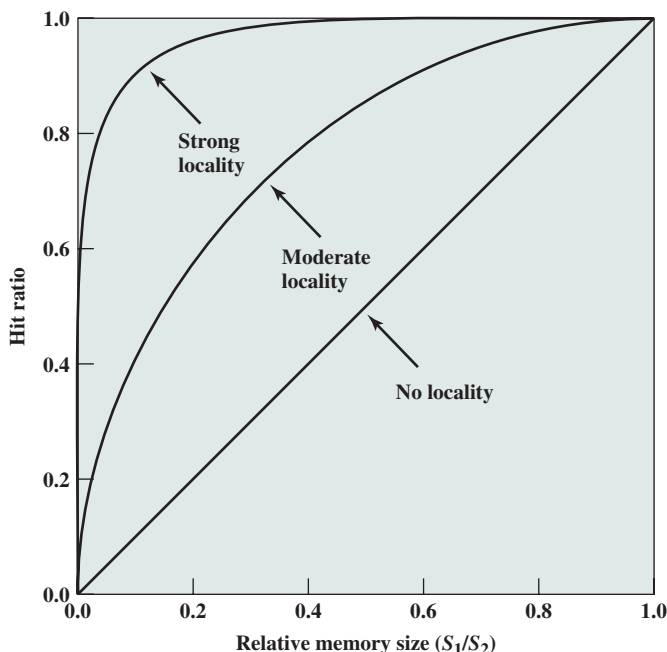


**Figure 4.22** Access Efficiency as a Function of Hit Ratio ( $r = T_2/T_1$ )

is about 5 to 15 times faster than main memory access time (i.e.,  $T_2/T_1$  is 5 to 15), and main memory access time is about 1000 times faster than disk access time ( $T_2/T_1 = 1000$ ). Thus, a hit ratio in the range of near 0.9 would seem to be needed to satisfy the performance requirement.

We can now phrase the question about relative memory size more exactly. Is a hit ratio of, say, 0.8 or better reasonable for  $S_1 \ll S_2$ ? This will depend on a number of factors, including the nature of the software being executed and the details of the design of the two-level memory. The main determinant is, of course, the degree of locality. Figure 4.23 suggests the effect that locality has on the hit ratio. Clearly, if M1 is the same size as M2, then the hit ratio will be 1.0: All of the items in M2 are always also stored in M1. Now suppose that there is no locality; that is, references are completely random. In that case the hit ratio should be a strictly linear function of the relative memory size. For example, if M1 is half the size of M2, then at any time half of the items from M2 are also in M1 and the hit ratio will be 0.5. In practice, however, there is some degree of locality in the references. The effects of moderate and strong locality are indicated in the figure. Note that Figure 4.23 is not derived from any specific data or model; the figure suggests the type of performance that is seen with various degrees of locality.

So if there is strong locality, it is possible to achieve high values of hit ratio even with relatively small upper-level memory size. For example, numerous studies have shown that rather small cache sizes will yield a hit ratio above 0.75 *regardless of the size of main memory* (e.g., [AGAR89], [PRZY88], [STRE83], and [SMIT82]). A cache in the range of 1K to 128K words is generally adequate, whereas main



**Figure 4.23** Hit Ratio as a Function of Relative Memory Size

memory is now typically in the gigabyte range. When we consider virtual memory and disk cache, we will cite other studies that confirm the same phenomenon, namely that a relatively small M1 yields a high value of hit ratio because of locality.

This brings us to the last question listed earlier: Does the relative size of the two memories satisfy the cost requirement? The answer is clearly yes. If we need only a relatively small upper-level memory to achieve good performance, then the average cost per bit of the two levels of memory will approach that of the cheaper lower-level memory.

Please note that with L2 cache, or even L2 and L3 caches, involved, analysis is much more complex. See [PEIR99] and [HAND98] for discussions.



# CHAPTER

# 5

## INTERNAL MEMORY

### **5.1 Semiconductor Main Memory**

- Organization
- DRAM and SRAM
- Types of ROM
- Chip Logic
- Chip Packaging
- Module Organization
- Interleaved Memory

### **5.2 Error Correction**

### **5.3 DDR DRAM**

- Synchronous DRAM
- DDR SDRAM

### **5.4 Flash Memory**

- Operation
- NOR and NAND Flash Memory

### **5.5 Newer Nonvolatile Solid-State Memory Technologies**

- STT-RAM
- PCRAM
- ReRAM

### **5.6 Key Terms, Review Questions, and Problems**

**LEARNING OBJECTIVES**

After studying this chapter, you should be able to:

- ◆ Present an overview of the principle types of semiconductor main memory.
- ◆ Understand the operation of a basic code that can detect and correct single-bit errors in 8-bit words.
- ◆ Summarize the properties of contemporary **DDR DRAM** organizations.
- ◆ Understand the difference between **NOR** and **NAND** flash memory.
- ◆ Present an overview of the newer nonvolatile solid-state memory technologies.

We begin this chapter with a survey of semiconductor main memory subsystems, including ROM, DRAM, and SRAM memories. Then we look at error control techniques used to enhance memory reliability. Following this, we look at more advanced DRAM architectures.

## 5.1 SEMICONDUCTOR MAIN MEMORY

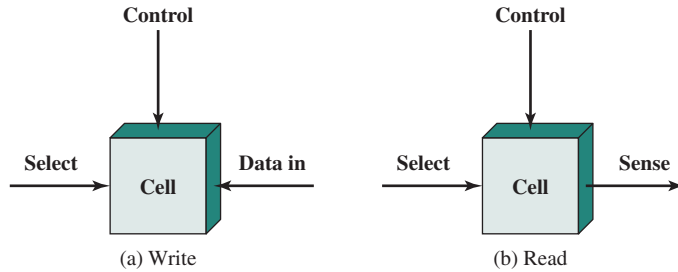
In earlier computers, the most common form of random-access storage for computer main memory employed an array of doughnut-shaped ferromagnetic loops referred to as *cores*. Hence, main memory was often referred to as *core*, a term that persists to this day. The advent of, and advantages of, microelectronics has long since vanquished the magnetic core memory. Today, the use of semiconductor chips for main memory is almost universal. Key aspects of this technology are explored in this section.

### Organization

The basic element of a **semiconductor memory** is the memory cell. Although a variety of electronic technologies are used, all semiconductor memory cells share certain properties:

- They exhibit two stable (or semistable) states, which can be used to represent binary 1 and 0.
- They are capable of being written into (at least once), to set the state.
- They are capable of being read to sense the state.

Figure 5.1 depicts the operation of a memory cell. Most commonly, the cell has three functional terminals capable of carrying an electrical signal. The select terminal, as the name suggests, selects a memory cell for a read or write operation. The control terminal indicates read or write. For writing, the other terminal provides an electrical signal that sets the state of the cell to 1 or 0. For reading, that terminal is used for output of the cell's state. The details of the internal organization, functioning, and timing of the memory cell depend on the specific integrated circuit technology used and are beyond the scope of this book, except for a brief summary. For our purposes, we will take it as given that individual cells can be selected for reading and writing operations.



**Figure 5.1** Memory Cell Operation

## DRAM and SRAM

All of the memory types that we will explore in this chapter are random access. That is, individual words of memory are directly accessed through wired-in addressing logic.

Table 5.1 lists the major types of semiconductor memory. The most common is referred to as **random-access memory (RAM)**. This is, in fact, a misuse of the term, because all of the types listed in the table are random access. One distinguishing characteristic of memory that is designated as RAM is that it is possible both to read data from the memory and to write new data into the memory easily and rapidly. Both the reading and writing are accomplished through the use of electrical signals.

The other distinguishing characteristic of traditional RAM is that it is volatile. A RAM must be provided with a constant power supply. If the power is interrupted, then the data are lost. Thus, RAM can be used only as temporary storage. The two traditional forms of RAM used in computers are DRAM and SRAM. Newer forms of RAM, discussed in Section 5.5, are nonvolatile.

**DYNAMIC RAM** RAM technology is divided into two technologies: dynamic and static. A **dynamic RAM (DRAM)** is made with cells that store data as charge on capacitors. The presence or absence of charge in a capacitor is interpreted as a binary 1 or 0. Because capacitors have a natural tendency to discharge, dynamic RAMs require periodic charge refreshing to maintain data storage. The term

**Table 5.1** Semiconductor Memory Types

| Memory Type                         | Category                  | Erasure                  | Write Mechanism | Volatility  |
|-------------------------------------|---------------------------|--------------------------|-----------------|-------------|
| Random-access memory (RAM)          | Read-write memory         | Electrically, byte-level | Electrically    | Volatile    |
| Read-only memory (ROM)              | Read-only memory          | Not possible             | Masks           | Nonvolatile |
| Programmable ROM (PROM)             |                           |                          | Electrically    |             |
| Erasable PROM (EPROM)               | UV light, chip-level      |                          |                 |             |
| Electrically Erasable PROM (EEPROM) | Electrically, byte-level  |                          |                 |             |
| Flash memory                        | Electrically, block-level |                          |                 |             |



*dynamic* refers to this tendency of the stored charge to leak away, even with power continuously applied.

Figure 5.2a is a typical DRAM structure for an individual cell that stores one bit. The address line is activated when the bit value from this cell is to be read or written. The transistor acts as a switch that is closed (allowing current to flow) if a voltage is applied to the address line and open (no current flows) if no voltage is present on the address line.

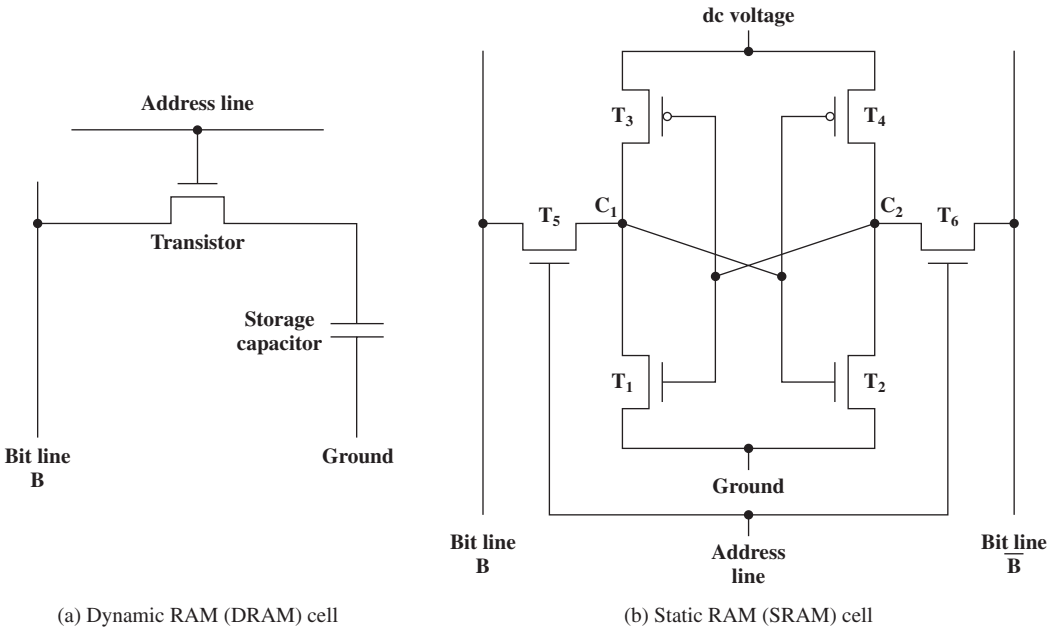
For the write operation, a voltage signal is applied to the bit line; a high voltage represents 1, and a low voltage represents 0. A signal is then applied to the address line, allowing a charge to be transferred to the capacitor.

For the read operation, when the address line is selected, the transistor turns on and the charge stored on the capacitor is fed out onto a bit line and to a sense amplifier. The sense amplifier compares the capacitor voltage to a reference value and determines if the cell contains a logic 1 or a logic 0. The readout from the cell discharges the capacitor, which must be restored to complete the operation.

Although the DRAM cell is used to store a single bit (0 or 1), it is essentially an analog device. The capacitor can store any charge value within a range; a threshold value determines whether the charge is interpreted as 1 or 0.

**STATIC RAM** In contrast, a **static RAM (SRAM)** is a digital device that uses the same logic elements used in the processor. In a SRAM, binary values are stored using traditional flip-flop logic-gate configurations (see Chapter 11 for a description of flip-flops). A static RAM will hold its data as long as power is supplied to it.

Figure 5.2b is a typical SRAM structure for an individual cell. Four transistors ( $T_1$ ,  $T_2$ ,  $T_3$ ,  $T_4$ ) are cross connected in an arrangement that produces a stable logic



**Figure 5.2** Typical Memory Cell Structures

state. In logic state 1, point  $C_1$  is high and point  $C_2$  is low; in this state,  $T_1$  and  $T_4$  are off and  $T_2$  and  $T_3$  are on.<sup>1</sup> In logic state 0, point  $C_1$  is low and point  $C_2$  is high; in this state,  $T_1$  and  $T_4$  are on and  $T_2$  and  $T_3$  are off. Both states are stable as long as the direct current (dc) voltage is applied. Unlike the DRAM, no refresh is needed to retain data.

As in the DRAM, the SRAM address line is used to open or close a switch. The address line controls two transistors ( $T_5$  and  $T_6$ ). When a signal is applied to this line, the two transistors are switched on, allowing a read or write operation. For a write operation, the desired bit value is applied to line B, while its complement is applied to line  $\bar{B}$ . This forces the four transistors ( $T_1, T_2, T_3, T_4$ ) into the proper state. For a read operation, the bit value is read from line B.

**SRAM VERSUS DRAM** Both static and dynamic RAMs are volatile; that is, power must be continuously supplied to the memory to preserve the bit values. A dynamic memory cell is simpler and smaller than a static memory cell. Thus, a DRAM is more dense (smaller cells = more cells per unit area) and less expensive than a corresponding SRAM. On the other hand, a DRAM requires the supporting refresh circuitry. For larger memories, the fixed cost of the refresh circuitry is more than compensated for by the smaller variable cost of DRAM cells. Thus, DRAMs tend to be favored for large memory requirements. A final point is that SRAMs are somewhat faster than DRAMs. Because of these relative characteristics, SRAM is used for cache memory (both on and off chip), and DRAM is used for main memory.

## Types of ROM

As the name suggests, a **read-only memory (ROM)** contains a permanent pattern of data that cannot be changed. A ROM is nonvolatile; that is, no power source is required to maintain the bit values in memory. While it is possible to read a ROM, it is not possible to write new data into it. An important application of ROMs is micro-programming, discussed in Part Four. Other potential applications include

- Library subroutines for frequently wanted functions
- System programs
- Function tables

For a modest-sized requirement, the advantage of ROM is that the data or program is permanently in main memory and need never be loaded from a secondary storage device.

A ROM is created like any other integrated circuit chip, with the data actually wired into the chip as part of the fabrication process. This presents two problems:

- The data insertion step includes a relatively large fixed cost, whether one or thousands of copies of a particular ROM are fabricated.
- There is no room for error. If one bit is wrong, the whole batch of ROMs must be thrown out.

When only a small number of ROMs with a particular memory content is needed, a less expensive alternative is the **programmable ROM (PROM)**. Like the

<sup>1</sup>The circles associated with  $T_3$  and  $T_4$  in Figure 5.2b indicate signal negation.

ROM, the PROM is **nonvolatile** and may be written into only once. For the PROM, the writing process is performed electrically and may be performed by a supplier or customer at a time later than the original chip fabrication. Special equipment is required for the writing or “programming” process. PROMs provide flexibility and convenience. The ROM remains attractive for high-volume production runs.

Another variation on read-only memory is the **read-mostly memory**, which is useful for applications in which read operations are far more frequent than write operations but for which nonvolatile storage is required. There are three common forms of read-mostly memory: EPROM, EEPROM, and flash memory.

The optically **erasable programmable read-only memory (EPROM)** is read and written electrically, as with PROM. However, before a write operation, all the storage cells must be erased to the same initial state by exposure of the packaged chip to ultraviolet radiation. Erasure is performed by shining an intense ultraviolet light through a window that is designed into the memory chip. This erasure process can be performed repeatedly; each erasure can take as much as 20 minutes to perform. Thus, the EPROM can be altered multiple times and, like the ROM and PROM, holds its data virtually indefinitely. For comparable amounts of storage, the EPROM is more expensive than PROM, but it has the advantage of the multiple update capability.

A more attractive form of read-mostly memory is **electrically erasable programmable read-only memory (EEPROM)**. This is a read-mostly memory that can be written into at any time without erasing prior contents; only the byte or bytes addressed are updated. The write operation takes considerably longer than the read operation, on the order of several hundred microseconds per byte. The EEPROM combines the advantage of nonvolatility with the flexibility of being updatable in place, using ordinary bus control, address, and data lines. EEPROM is more expensive than EPROM and also is less dense, supporting fewer bits per chip.

Another form of semiconductor memory is **flash memory** (so named because of the speed with which it can be reprogrammed). First introduced in the mid-1980s, flash memory is intermediate between EPROM and EEPROM in both cost and functionality. Like EEPROM, flash memory uses an electrical erasing technology. An entire flash memory can be erased in one or a few seconds, which is much faster than EPROM. In addition, it is possible to erase just blocks of memory rather than an entire chip. Flash memory gets its name because the microchip is organized so that a section of memory cells are erased in a single action or “flash.” However, flash memory does not provide byte-level erasure. Like EPROM, flash memory uses only one transistor per bit, and so achieves the high density (compared with EEPROM) of EPROM.

## Chip Logic

As with other integrated circuit products, semiconductor memory comes in packaged chips (Figure 1.11). Each chip contains an array of memory cells.

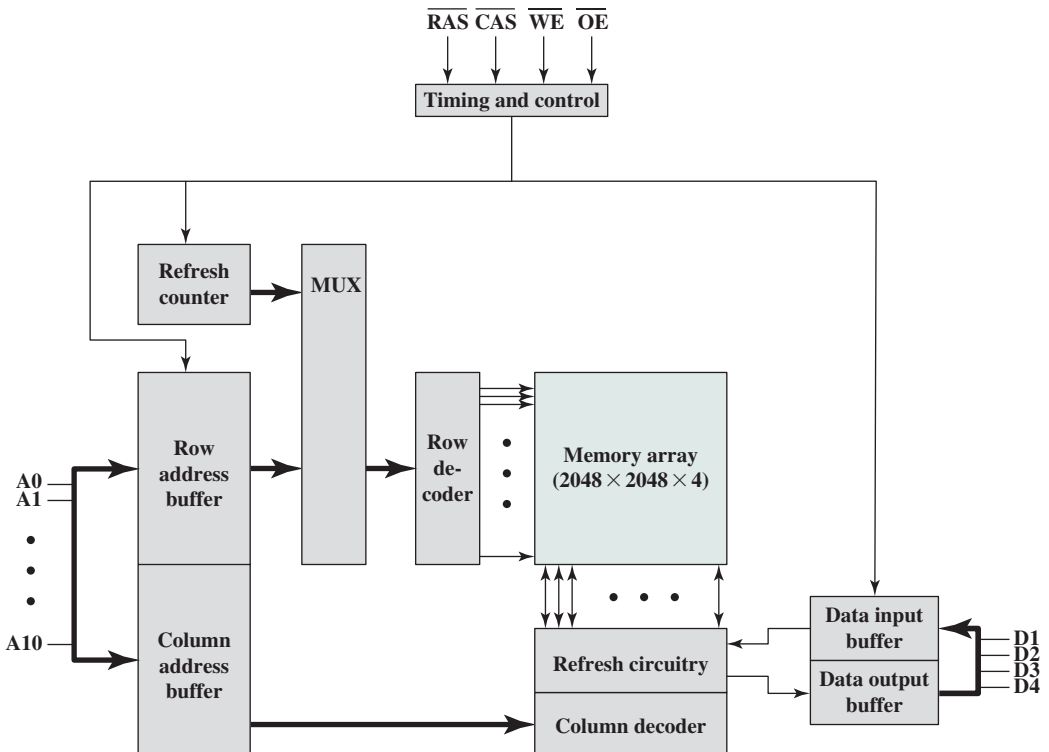
In the memory hierarchy as a whole, we saw that there are trade-offs among speed, density, and cost. These trade-offs also exist when we consider the organization of memory cells and functional logic on a chip. For semiconductor memories, one of the key design issues is the number of bits of data that may be read/written at a time. At one extreme is an organization in which the physical arrangement of cells in the array is the same as the logical arrangement (as perceived by the processor) of words in memory. The array is organized into  $W$  words of  $B$  bits each.

For example, a 16-Mbit chip could be organized as 1M 16-bit words. At the other extreme is the so-called 1-bit-per-chip organization, in which data are read/written one bit at a time. We will illustrate memory chip organization with a DRAM; ROM organization is similar, though simpler.

Figure 5.3 shows a typical organization of a 16-Mbit DRAM. In this case, 4 bits are read or written at a time. Logically, the memory array is organized as four square arrays of 2048 by 2048 elements. Various physical arrangements are possible. In any case, the elements of the array are connected by both horizontal (row) and vertical (column) lines. Each horizontal line connects to the Select terminal of each cell in its row; each vertical line connects to the Data-In/Sense terminal of each cell in its column.

Address lines supply the address of the word to be selected. A total of  $\log_2 W$  lines are needed. In our example, 11 address lines are needed to select one of 2048 rows. These 11 lines are fed into a row decoder, which has 11 lines of input and 2048 lines for output. The logic of the decoder activates a single one of the 2048 outputs depending on the bit pattern on the 11 input lines ( $2^{11} = 2048$ ).

An additional 11 address lines select one of 2048 columns of 4 bits per column. Four data lines are used for the input and output of 4 bits to and from a data buffer. On input (write), the bit driver of each bit line is activated for a 1 or 0 according to the value of the corresponding data line. On output (read), the value of each bit line is passed through a sense amplifier and presented to the data lines. The row line selects which row of cells is used for reading or writing.



**Figure 5.3** Typical 16-Mbit DRAM ( $4\text{M} \times 4$ )

Because only 4 bits are read/written to this DRAM, there must be multiple DRAMs connected to the memory controller to read/write a word of data to the bus.

Note that there are only 11 address lines (A0–A10), half the number you would expect for a  $2048 \times 2048$  array. This is done to save on the number of pins. The 22 required address lines are passed through select logic external to the chip and multiplexed onto the 11 address lines. First, 11 address signals are passed to the chip to define the row address of the array, and then the other 11 address signals are presented for the column address. These signals are accompanied by row address select ( $\overline{\text{RAS}}$ ) and column address select ( $\overline{\text{CAS}}$ ) signals to provide timing to the chip.

The write enable ( $\overline{\text{WE}}$ ) and output enable ( $\overline{\text{OE}}$ ) pins determine whether a write or read operation is performed. Two other pins, not shown in Figure 5.3, are ground ( $V_{ss}$ ) and a voltage source ( $V_{cc}$ ).

As an aside, multiplexed addressing plus the use of square arrays result in a quadrupling of memory size with each new generation of memory chips. One more pin devoted to addressing doubles the number of rows and columns, and so the size of the chip memory grows by a factor of 4.

Figure 5.3 also indicates the inclusion of refresh circuitry. All DRAMs require a refresh operation. A simple technique for refreshing is, in effect, to disable the DRAM chip while all data cells are refreshed. The refresh counter steps through all of the row values. For each row, the output lines from the refresh counter are supplied to the row decoder and the RAS line is activated. The data are read out and written back into the same location. This causes each cell in the row to be refreshed.

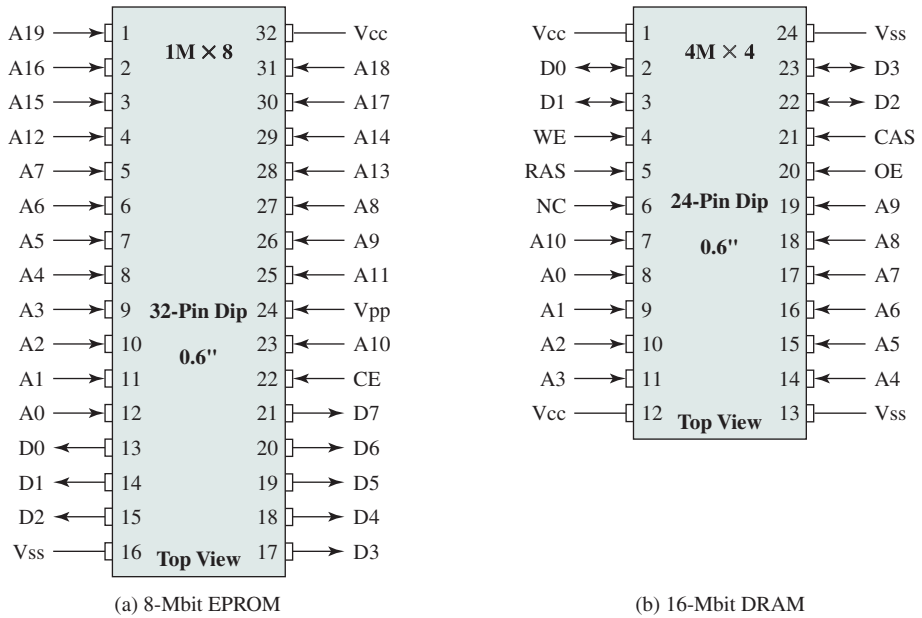
## Chip Packaging

As was mentioned in Chapter 2, an integrated circuit is mounted on a package that contains pins for connection to the outside world.

Figure 5.4a shows an example EPROM package, which is an 8-Mbit chip organized as  $1\text{M} \times 8$ . In this case, the organization is treated as a one-word-per-chip package. The package includes 32 pins, which is one of the standard chip package sizes. The pins support the following signal lines:

- The address of the word being accessed. For 1M words, a total of 20 ( $2^{20} = 1\text{M}$ ) pins are needed (A0–A19).
- The data to be read out, consisting of 8 lines (D0–D7).
- The power supply to the chip ( $V_{cc}$ ).
- A ground pin ( $V_{ss}$ ).
- A chip enable (CE) pin. Because there may be more than one memory chip, each of which is connected to the same address bus, the CE pin is used to indicate whether or not the address is valid for this chip. The CE pin is activated by logic connected to the higher-order bits of the address bus (i.e., address bits above A19). The use of this signal is illustrated presently.
- A program voltage ( $V_{pp}$ ) that is supplied during programming (write operations).

A typical DRAM pin configuration is shown in Figure 5.4b, for a 16-Mbit chip organized as  $4\text{M} \times 4$ . There are several differences from a ROM chip. Because a RAM can be updated, the data pins are input/output. The write enable (WE) and output enable (OE) pins indicate whether this is a write or read operation.



**Figure 5.4** Typical Memory Package Pins and Signals

Because the DRAM is accessed by row and column, and the address is multiplexed, only 11 address pins are needed to specify the 4M row/column combinations ( $2^{11} \times 2^{11} = 2^{22} = 4\text{M}$ ). The functions of the row address select (RAS) and column address select (CAS) pins were discussed previously. Finally, the no connect (NC) pin is provided so that there are an even number of pins.

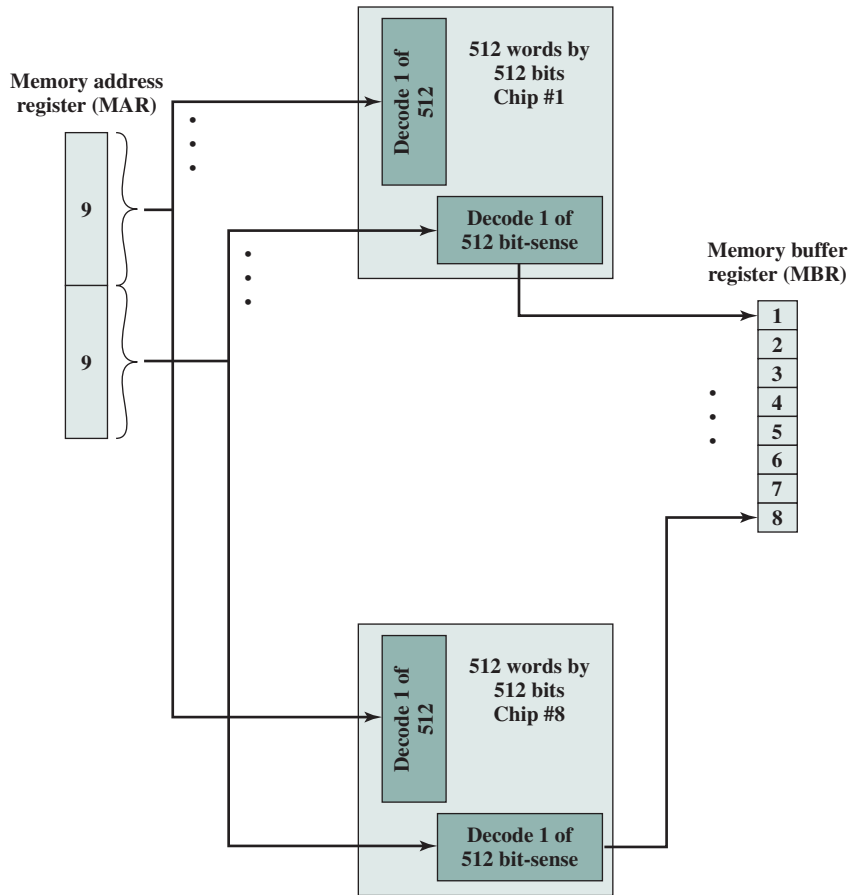
## Module Organization

If a RAM chip contains only one bit per word, then clearly we will need at least a number of chips equal to the number of bits per word. As an example, Figure 5.5 shows how a memory module consisting of 256K 8-bit words could be organized. For 256K words, an 18-bit address is needed and is supplied to the module from some external source (e.g., the address lines of a bus to which the module is attached). The address is presented to 8  $256\text{K} \times 1\text{-bit}$  chips, each of which provides the input/output of one bit.

This organization works as long as the size of memory equals the number of bits per chip. In the case in which larger memory is required, an array of chips is needed. Figure 5.6 shows the possible organization of a memory consisting of 1M word by 8 bits per word. In this case, we have four columns of chips, each column containing 256K words arranged as in Figure 5.5. For 1M word, 20 address lines are needed. The 18 least significant bits are routed to all 32 modules. The high-order 2 bits are input to a group select logic module that sends a chip enable signal to one of the four columns of modules.

## Interleaved Memory

Main memory is composed of a collection of DRAM memory chips. A number of chips can be grouped together to form a *memory bank*. It is possible to organize the memory



**Figure 5.5** 256-KByte Memory Organization

banks in a way known as interleaved memory. Each bank is independently able to service a memory read or write request, so that a system with  $K$  banks can service  $K$  requests simultaneously, increasing memory read or write rates by a factor of  $K$ . If consecutive words of memory are stored in different banks, then the transfer of a block of memory is speeded up. Appendix G explores the topic of interleaved memory.



**Interleaved Memory Simulator**

**5.2 ERROR CORRECTION**

A semiconductor memory system is subject to errors. These can be categorized as hard failures and soft errors. A **hard failure** is a permanent physical defect so that the memory cell or cells affected cannot reliably store data but become stuck at 0 or 1 or

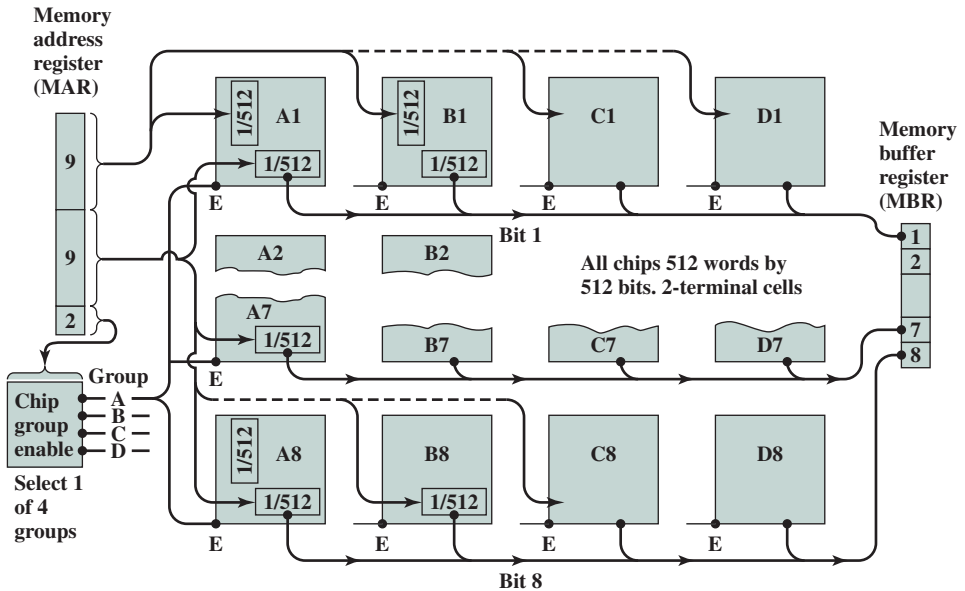


Figure 5.6 1-MB Memory Organization

switch erratically between 0 and 1. Hard errors can be caused by harsh environmental abuse, manufacturing defects, and wear. A **soft error** is a random, nondestructive event that alters the contents of one or more memory cells without damaging the memory. Soft errors can be caused by power supply problems or alpha particles. These particles result from radioactive decay and are distressingly common because radioactive nuclei are found in small quantities in nearly all materials. Both hard and soft errors are clearly undesirable, and most modern main memory systems include logic for both detecting and correcting errors.

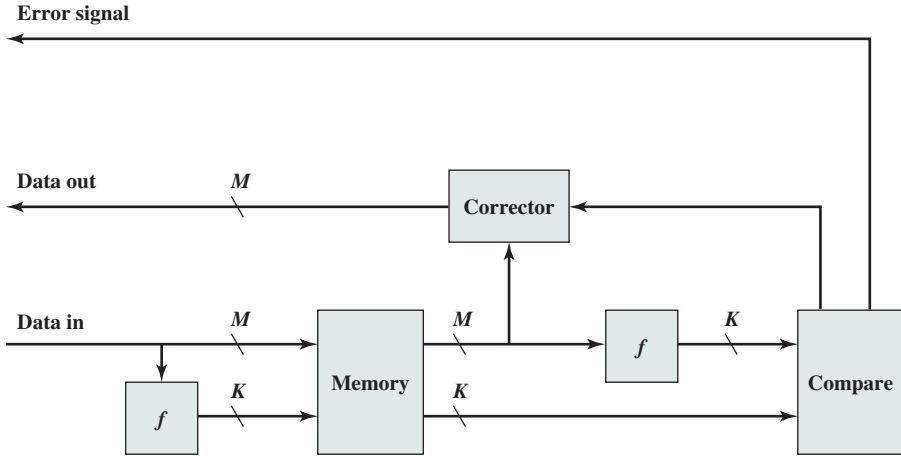
Figure 5.7 illustrates in general terms how the process is carried out. When data are to be written into memory, a calculation, depicted as a function  $f$ , is performed on the data to produce a code. Both the code and the data are stored. Thus, if an  $M$ -bit word of data is to be stored and the code is of length  $K$  bits, then the actual size of the stored word is  $M + K$  bits.

When the previously stored word is read out, the code is used to detect and possibly correct errors. A new set of  $K$  code bits is generated from the  $M$  data bits and compared with the fetched code bits. The comparison yields one of three results:

- No errors are detected. The fetched data bits are sent out.
- An error is detected, and it is possible to correct the error. The data bits plus **error correction** bits are fed into a corrector, which produces a corrected set of  $M$  bits to be sent out.
- An error is detected, but it is not possible to correct it. This condition is reported.

Codes that operate in this fashion are referred to as **error-correcting codes**. A code is characterized by the number of bit errors in a word that it can correct and detect.





**Figure 5.7** Error-Correcting Code Function

The simplest of the error-correcting codes is the **Hamming code** devised by Richard Hamming at Bell Laboratories. Figure 5.8 uses Venn diagrams to illustrate the use of this code on 4-bit words ( $M = 4$ ). With three intersecting circles, there are seven compartments. We assign the 4 data bits to the inner compartments (Figure 5.8a). The remaining compartments are filled with what are called *parity bits*. Each parity bit is chosen so that the total number of 1s in its circle is even (Figure 5.8b). Thus, because circle A includes three data 1s, the parity bit in that circle is set to 1. Now, if an error changes one of the data bits (Figure 5.8c), it is easily found. By checking the parity bits, discrepancies are found in circle A and circle C but not in circle B. Only one of the seven compartments is in A and C but not B (Figure 5.8d). The error can therefore be corrected by changing that bit.

To clarify the concepts involved, we will develop a code that can detect and correct single-bit errors in 8-bit words.

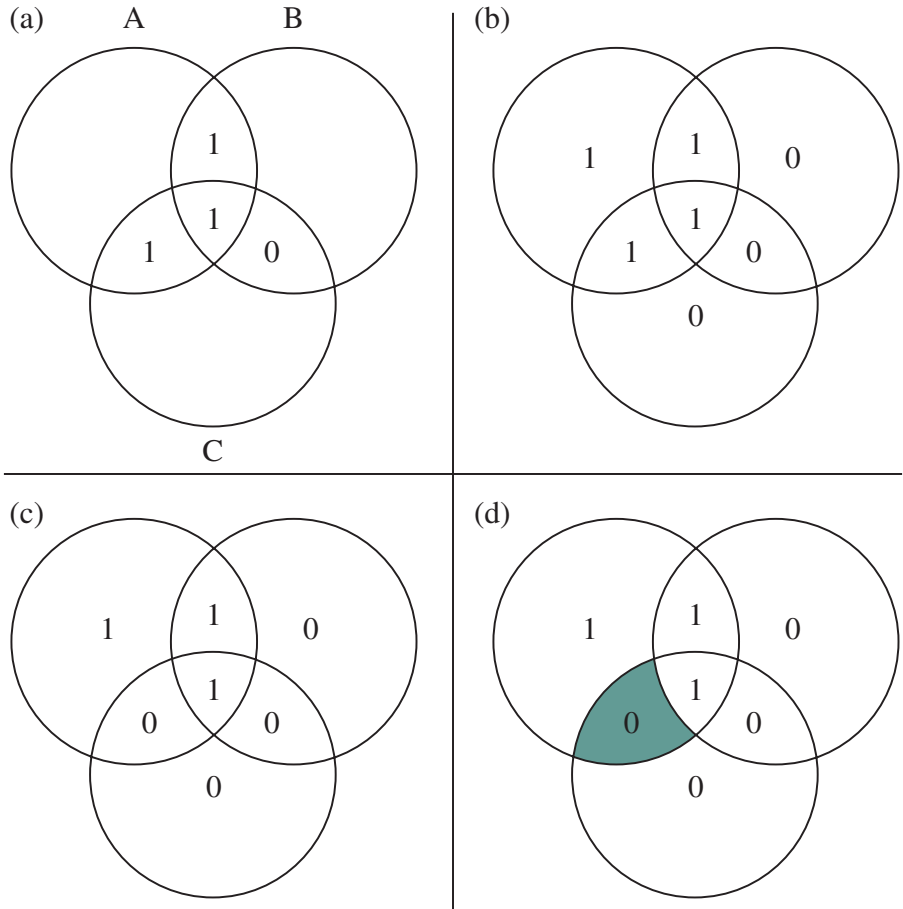
To start, let us determine how long the code must be. Referring to Figure 5.7, the comparison logic receives as input two  $K$ -bit values. A bit-by-bit comparison is done by taking the exclusive-OR of the two inputs. The result is called the *syndrome word*. Thus, each bit of the **syndrome** is 0 or 1 according to if there is or is not a match in that bit position for the two inputs.

The syndrome word is therefore  $K$  bits wide and has a range between 0 and  $2^K - 1$ . The value 0 indicates that no error was detected, leaving  $2^K - 1$  values to indicate, if there is an error, which bit was in error. Now, because an error could occur on any of the  $M$  data bits or  $K$  check bits, we must have

$$2^K - 1 \geq M + K$$

This inequality gives the number of bits needed to correct a single bit error in a word containing  $M$  data bits. For example, for a word of 8 data bits ( $M = 8$ ), we have

- $K = 3: 2^3 - 1 < 8 + 3$
- $K = 4: 2^4 - 1 > 8 + 4$



**Figure 5.8** Hamming Error-Correcting Code

Thus, eight data bits require four check bits. The first three columns of Table 5.2 lists the number of check bits required for various data word lengths.

For convenience, we would like to generate a 4-bit syndrome for an 8-bit data word with the following characteristics:

- If the syndrome contains all 0s, no error has been detected.
- If the syndrome contains one and only one bit set to 1, then an error has occurred in one of the 4 check bits. No correction is needed.
- If the syndrome contains more than one bit set to 1, then the numerical value of the syndrome indicates the position of the data bit in error. This data bit is inverted for correction.

To achieve these characteristics, the data and check bits are arranged into a 12-bit word as depicted in Figure 5.9. The bit positions are numbered from 1 to 12. Those bit positions whose position numbers are powers of 2 are designated as check

**Table 5.2** Increase in Word Length with Error Correction

| Data Bits | Single-Error Correction |            | Single-Error Correction/<br>Double-Error Detection |            |
|-----------|-------------------------|------------|----------------------------------------------------|------------|
|           | Check Bits              | % Increase | Check Bits                                         | % Increase |
| 8         | 4                       | 50.0       | 5                                                  | 62.5       |
| 16        | 5                       | 31.25      | 6                                                  | 37.5       |
| 32        | 6                       | 18.75      | 7                                                  | 21.875     |
| 64        | 7                       | 10.94      | 8                                                  | 12.5       |
| 128       | 8                       | 6.25       | 9                                                  | 7.03       |
| 256       | 9                       | 3.52       | 10                                                 | 3.91       |

bits. The check bits are calculated as follows, where the symbol  $\oplus$  designates the exclusive-OR operation:

$$\begin{aligned} C1 &= D1 \oplus D2 \oplus \quad \quad D4 \oplus D5 \oplus \quad \quad D7 \\ C2 &= D1 \oplus \quad \quad D3 \oplus D4 \oplus \quad \quad D6 \oplus D7 \\ C4 &= \quad \quad D2 \oplus D3 \oplus D4 \oplus \quad \quad \quad D8 \\ C8 &= \quad \quad \quad \oplus D5 \oplus D6 \oplus D7 \oplus D8 \end{aligned}$$

Each check bit operates on every data bit whose position number contains a 1 in the same bit position as the position number of that check bit. Thus, data bit positions 3, 5, 7, 9, and 11 (D1, D2, D4, D5, D7) all contain a 1 in the least significant bit of their position number as does C1; bit positions 3, 6, 7, 10, and 11 all contain a 1 in the second bit position, as does C2; and so on. Looked at another way, bit position  $n$  is checked by those bits  $C_i$  such that  $\sum_i i = n$ . For example, position 7 is checked by bits in position 4, 2, and 1; and  $7 = 4 + 2 + 1$ .

Let us verify that this scheme works with an example. Assume that the 8-bit input word is 00111001, with data bit D1 in the rightmost position. The calculations are as follows:

$$\begin{aligned} C1 &= 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1 \\ C2 &= 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1 \\ C4 &= 0 \oplus 0 \oplus 1 \oplus 0 = 1 \\ C8 &= 1 \oplus 1 \oplus 0 \oplus 0 = 0 \end{aligned}$$

| Bit position    | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    |
|-----------------|------|------|------|------|------|------|------|------|------|------|------|------|
| Position number | 1100 | 1011 | 1010 | 1001 | 1000 | 0111 | 0110 | 0101 | 0100 | 0011 | 0010 | 0001 |
| Data bit        | D8   | D7   | D6   | D5   |      | D4   | D3   | D2   |      | D1   |      |      |
| Check bit       |      |      |      |      | C8   |      |      |      | C4   |      | C2   | C1   |

**Figure 5.9** Layout of Data Bits and Check Bits

Suppose now that data bit 3 sustains an error and is changed from 0 to 1. When the check bits are recalculated, we have

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C4 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

When the new check bits are compared with the old check bits, the syndrome word is formed:

|          |    |    |    |    |
|----------|----|----|----|----|
|          | C8 | C4 | C2 | C1 |
|          | 0  | 1  | 1  | 1  |
| $\oplus$ | 0  | 0  | 0  | 1  |
|          | 0  | 1  | 1  | 0  |

The result is 0110, indicating that bit position 6, which contains data bit 3, is in error.

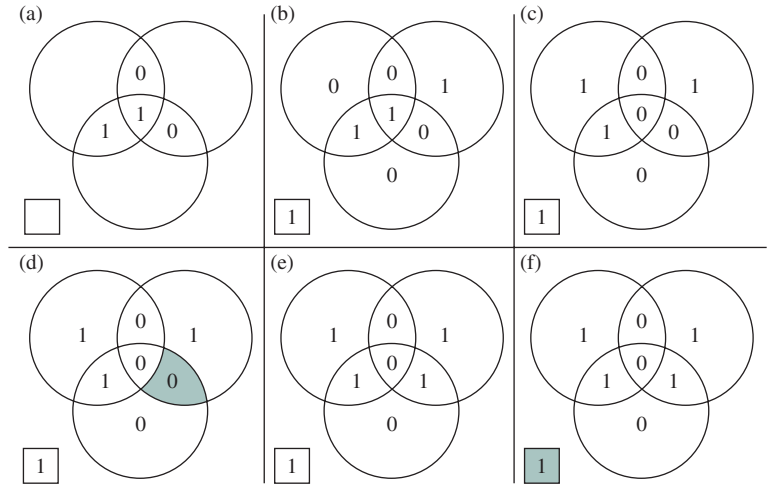
Figure 5.10 illustrates the preceding calculation. The data and check bits are positioned properly in the 12-bit word. Four of the data bits have a value 1 (shaded in the table), and their bit position values are XORed to produce the Hamming code 0111, which forms the four check digits. The entire block that is stored is 001101001111. Suppose now that data bit 3, in bit position 6, sustains an error and is changed from 0 to 1. The resulting block is 001101101111, with a Hamming code of 0001. An XOR of the Hamming code and all of the bit position values for nonzero data bits results in 0110. The nonzero result detects an error and indicates that the error is in bit position 6.

The code just described is known as a **single-error-correcting (SEC) code**. More commonly, semiconductor memory is equipped with a **single-error-correcting, double-error-detecting (SEC-DED) code**. As Table 5.2 shows, such codes require one additional bit compared with SEC codes.

Figure 5.11 illustrates how such a code works, again with a 4-bit data word. The sequence shows that if two errors occur (Figure 5.11c), the checking procedure goes astray (d) and worsens the problem by creating a third error (e). To overcome

| Bit position    | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    |
|-----------------|------|------|------|------|------|------|------|------|------|------|------|------|
| Position number | 1100 | 1011 | 1010 | 1001 | 1000 | 0111 | 0110 | 0101 | 0100 | 0011 | 0010 | 0001 |
| Data bit        | D8   | D7   | D6   | D5   |      | D4   | D3   | D2   |      | D1   |      |      |
| Check bit       |      |      |      |      | C8   |      |      |      | C4   |      | C2   | C1   |
| Word stored as  | 0    | 0    | 1    | 1    | 0    | 1    | 0    | 0    | 1    | 1    | 1    | 1    |
| Word fetched as | 0    | 0    | 1    | 1    | 0    | 1    | 1    | 0    | 1    | 1    | 1    | 1    |
| Position number | 1100 | 1011 | 1010 | 1001 | 1000 | 0111 | 0110 | 0101 | 0100 | 0011 | 0010 | 0001 |
| Check bit       |      |      |      |      | 0    |      |      |      | 0    |      | 0    | 1    |

**Figure 5.10** Check Bit Calculation



**Figure 5.11** Hamming SEC-DEC Code

the problem, an eighth bit is added that is set so that the total number of 1s in the diagram is even. The extra parity bit catches the error (f).

An error-correcting code enhances the reliability of the memory at the cost of added complexity. With a 1-bit-per-chip organization, an SEC-DED code is generally considered adequate. For example, the IBM 30xx implementations used an 8-bit SEC-DED code for each 64 bits of data in main memory. Thus, the size of main memory is actually about 12% larger than is apparent to the user. The VAX computers used a 7-bit SEC-DED for each 32 bits of memory, for a 22% overhead. Contemporary DRAM systems may have anywhere from 7% to 20% overhead [SHAR03].

**5.3 DDR DRAM**

As discussed in Chapter 1, one of the most critical system bottlenecks when using high-performance processors is the interface to internal main memory. This interface is the most important pathway in the entire computer system. The basic building block of main memory remains the DRAM chip, as it has for decades; until recently, there had been no significant changes in DRAM architecture since the early 1970s. The traditional DRAM chip is constrained both by its internal architecture and by its interface to the processor's memory bus.

We have seen that one attack on the performance problem of DRAM main memory has been to insert one or more levels of high-speed SRAM cache between the DRAM main memory and the processor. But SRAM is much costlier than DRAM, and expanding cache size beyond a certain point yields diminishing returns.

In recent years, a number of enhancements to the basic DRAM architecture have been explored. The schemes that currently dominate the market are SDRAM and DDR-DRAM. We examine each of these in turn.

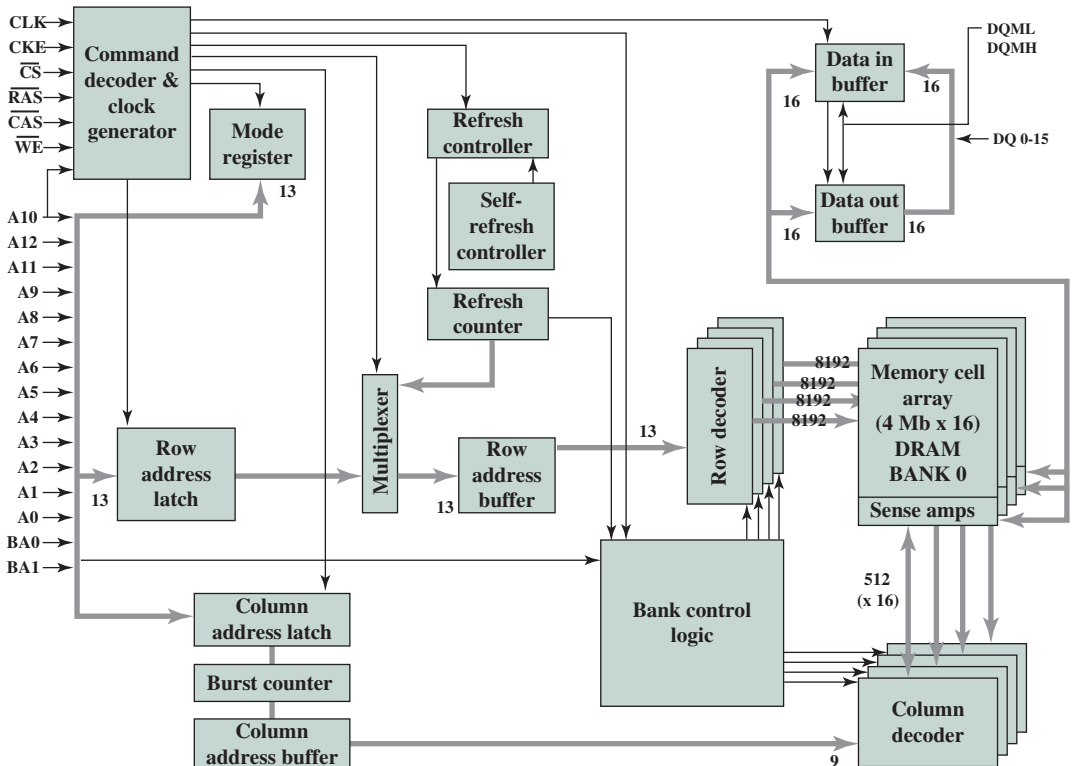
## Synchronous DRAM

One of the most widely used forms of DRAM is the **synchronous DRAM (SDRAM)**. Unlike the traditional DRAM, which is asynchronous, the SDRAM exchanges data with the processor synchronized to an external clock signal and running at the full speed of the processor/memory bus without imposing wait states.

In a typical DRAM, the processor presents addresses and control levels to the memory, indicating that a set of data at a particular location in memory should be either read from or written into the DRAM. After a delay, the access time, the DRAM either writes or reads the data. During the access-time delay, the DRAM performs various internal functions, such as activating the high capacitance of the row and column lines, sensing the data, and routing the data out through the output buffers. The processor must simply wait through this delay, slowing system performance.

With synchronous access, the DRAM moves data in and out under control of the system clock. The processor or other master issues the instruction and address information, which is latched by the DRAM. The DRAM then responds after a set number of clock cycles. Meanwhile, the master can safely do other tasks while the SDRAM is processing the request.

Figure 5.12 shows the internal logic of a typical 256-Mb SDRAM typical of SDRAM organization, and Table 5.3 defines the various pin assignments. The



**Figure 5.12** 256-Mb Synchronous Dynamic RAM (SDRAM)

**Table 5.3** SDRAM Pin Assignments

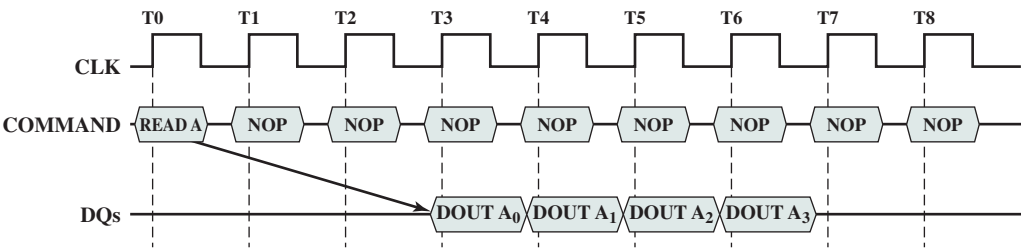
|                         |                       |
|-------------------------|-----------------------|
| A0 to A13               | Address inputs        |
| BA0, BA1                | Bank address lines    |
| CLK                     | Clock input           |
| CKE                     | Clock enable          |
| $\overline{\text{CS}}$  | Chip select           |
| $\overline{\text{RAS}}$ | Row address strobe    |
| $\overline{\text{CAS}}$ | Column address strobe |
| $\overline{\text{WE}}$  | Write enable          |
| DQ0 to DQ7              | Data input/output     |
| DQM                     | Data mask             |

SDRAM employs a burst mode to eliminate the address setup time and row and column line precharge time after the first access. In burst mode, a series of data bits can be clocked out rapidly after the first bit has been accessed. This mode is useful when all the bits to be accessed are in sequence and in the same row of the array as the initial access. In addition, the SDRAM has a multiple-bank internal architecture that improves opportunities for on-chip parallelism.

The mode register and associated control logic is another key feature differentiating SDRAMs from conventional DRAMs. It provides a mechanism to customize the SDRAM to suit specific system needs. The mode register specifies the burst length, which is the number of separate units of data synchronously fed onto the bus. The register also allows the programmer to adjust the latency between receipt of a read request and the beginning of data transfer.

The SDRAM performs best when it is transferring large blocks of data sequentially, such as for applications like word processing, spreadsheets, and multimedia.

Figure 5.13 shows an example of SDRAM operation. In this case, the burst length is 4 and the latency is 2. The burst read command is initiated by having  $\overline{\text{CS}}$  and  $\overline{\text{CAS}}$  low while holding  $\overline{\text{RAS}}$  and  $\overline{\text{WE}}$  high at the rising edge of the clock. The address inputs determine the starting column address for the burst, and the mode register sets the type of burst (sequential or interleave) and the burst length (1, 2, 4, 8, full page). The delay from the start of the command to when the data from the first cell appears on the outputs is equal to the value of the  $\overline{\text{CAS}}$  latency that is set in the mode register.



**Figure 5.13** SDRAM Read Timing (burst length = 4,  $\overline{\text{CAS}}$  latency = 2)

## DDR SDRAM

Although SDRAM is a significant improvement on asynchronous RAM, it still has shortcomings that unnecessarily limit that I/O data rate that can be achieved. To address these shortcomings a newer version of SDRAM, referred to as double-data-rate DRAM (DDR DRAM) provides several features that dramatically increase the data rate. DDR DRAM was developed by the JEDEC Solid State Technology Association, the Electronic Industries Alliance's semiconductor-engineering-standardization body. Numerous companies make DDR chips, which are widely used in desktop computers and servers.

DDR achieves higher data rates in three ways. First, the data transfer is synchronized to both the rising and falling edge of the clock, rather than just the rising edge. This doubles the data rate; hence the term *double data rate*. Second, DDR uses higher clock rate on the bus to increase the transfer rate. Third, a buffering scheme is used, as explained subsequently.

JEDEC has thus far defined four generations of the DDR technology (Table 5.4). The initial DDR version makes use of a 2-bit prefetch buffer. The prefetch buffer is a memory cache located on the SDRAM chip. It enables the SDRAM chip to preposition bits to be placed on the data bus as rapidly as possible. The DDR I/O bus uses the same clock rate as the memory chip, but because it can handle two bits per cycle, it achieves a data rate that is double the clock rate. The 2-bit prefetch buffer enables the SDRAM chip to keep up with the I/O bus.

To understand the operation of the prefetch buffer, we need to look at it from the point of view of a word transfer. The prefetch buffer size determines how many words of data are fetched (across multiple SDRAM chips) every time a column command is performed with DDR memories. Because the core of the DRAM is much slower than the interface, the difference is bridged by accessing information in parallel and then serializing it out the interface through a multiplexor (MUX). Thus, DDR prefetches two words, which means that every time a read or a write operation is performed, it is performed on two words of data, and bursts out of, or into, the SDRAM over one clock cycle on both clock edges for a total of two consecutive operations. As a result, the DDR I/O interface is twice as fast as the SDRAM core.

Although each new generation of SDRAM results in much greater capacity, the core speed of the SDRAM has not changed significantly from generation to generation. To achieve greater data rates than those afforded by the rather modest increases in SDRAM clock rate, JEDEC increased the buffer size. For DDR2, a 4-bit buffer is used, allowing for words to be transferred in parallel, increasing the effective data rate by a factor of 4. For DDR3, an 8-bit buffer is used and a factor of 8 speedup is achieved (Figure 5.14).

**Table 5.4** DDR Characteristics

|                                  | DDR1    | DDR2     | DDR3     | DDR4      |
|----------------------------------|---------|----------|----------|-----------|
| Prefetch buffer (bits)           | 2       | 4        | 8        | 8         |
| Voltage level (V)                | 2.5     | 1.8      | 1.5      | 1.2       |
| Front side bus data rates (Mbps) | 200–400 | 400–1066 | 800–2133 | 2133–4266 |



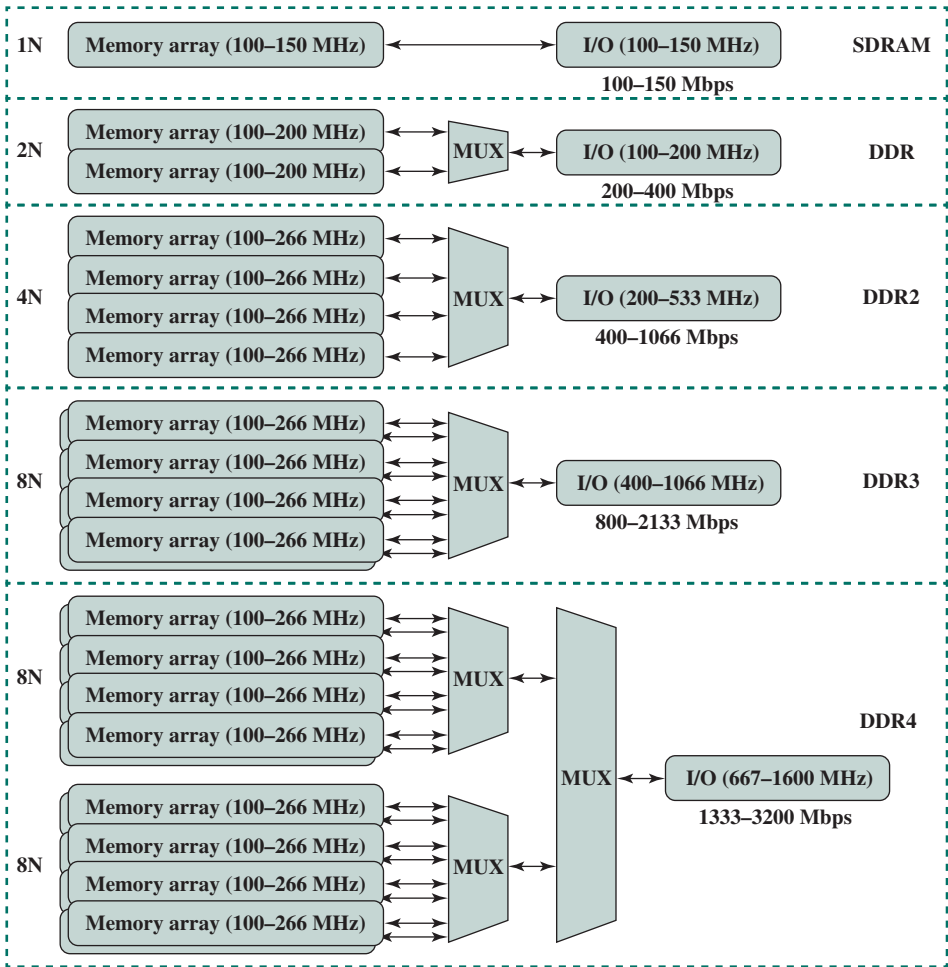


Figure 5.14 DDR Generations

The downside to the prefetch is that it effectively determines the minimum burst length for the SDRAMs. For example, it is very difficult to have an efficient burst length of four words with DDR3's prefetch of eight. Accordingly, the JEDEC designers chose not to increase the buffer size to 16 bits for DDR4, but rather to introduce the concept of a **bank group** [ALLA13]. Bank groups are separate entities such that they allow a column cycle to complete within a bank group, but that column cycle does not impact what is happening in another bank group. Thus, two prefetches of eight can be operating in parallel in the two bank groups. This arrangement keeps the prefetch buffer size the same as for DDR3, while increasing performance as if the prefetch is larger.

Figure 5.14 shows a configuration with two bank groups. With DDR4, up to 4 bank groups can be used.

## 5.4 FLASH MEMORY

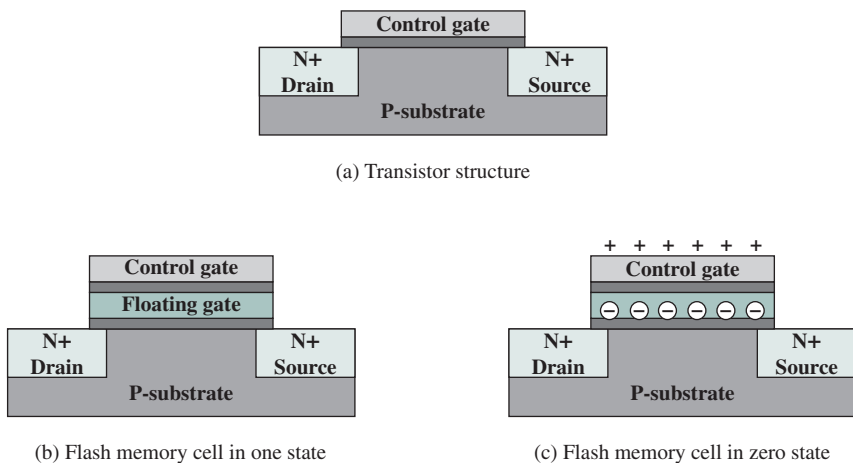
Another form of semiconductor memory is flash memory. Flash memory is used both for internal memory and external memory applications. Here, we provide a technical overview and look at its use for internal memory.

First introduced in the mid-1980s, flash memory is intermediate between EPROM and EEPROM in both cost and functionality. Like EEPROM, flash memory uses an electrical erasing technology. An entire flash memory can be erased in one or a few seconds, which is much faster than EPROM. In addition, it is possible to erase just blocks of memory rather than an entire chip. Flash memory gets its name because the microchip is organized so that a section of memory cells are erased in a single action or “flash.” However, flash memory does not provide byte-level erasure. Like EPROM, flash memory uses only one transistor per bit, and so achieves the high density (compared with EEPROM) of EPROM.

### Operation

Figure 5.15 illustrates the basic operation of a flash memory. For comparison, Figure 5.15a depicts the operation of a transistor. Transistors exploit the properties of semiconductors so that a small voltage applied to the gate can be used to control the flow of a large current between the source and the drain.

In a flash memory cell, a second gate—called a floating gate, because it is insulated by a thin oxide layer—is added to the transistor. Initially, the floating gate does not interfere with the operation of the transistor (Figure 5.15b). In this state, the cell is deemed to represent binary 1. Applying a large voltage across the oxide layer causes electrons to tunnel through it and become trapped on the floating gate, where they remain even if the power is disconnected (Figure 5.15c). In this state, the cell is deemed to represent binary 0. The state of the cell can be read by using external circuitry to test whether the transistor is working or not. Applying a large voltage in the opposite direction removes the electrons from the floating gate, returning to a state of binary 1.



**Figure 5.15** Flash Memory Operation

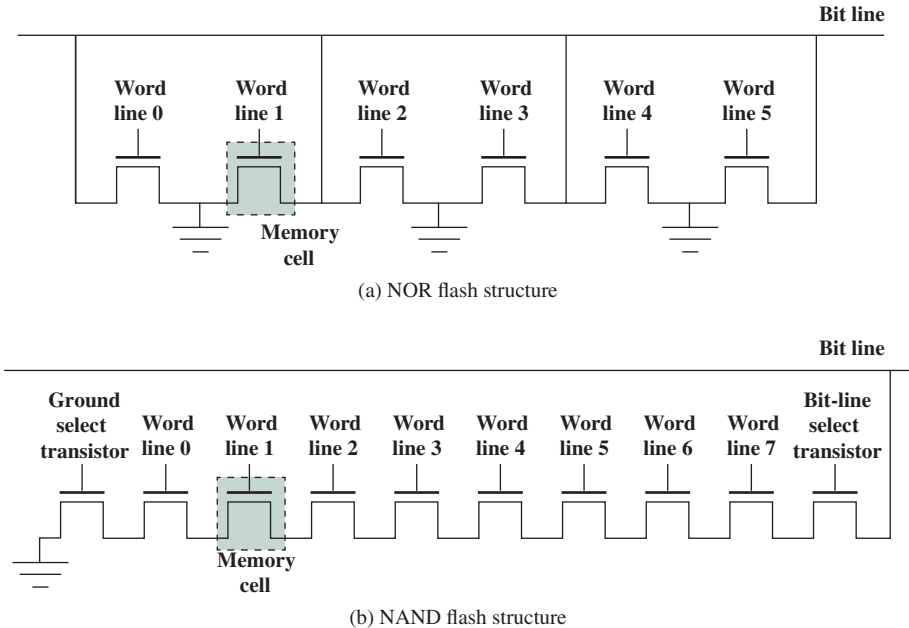
An important characteristic of flash memory is that it is persistent memory, which means that it retains data when there is no power applied to the memory. Thus, it is useful for secondary (external) storage, and as an alternative to random access memory in computers.

### NOR and NAND Flash Memory

There are two distinctive types of flash memory, designated as NOR and NAND (Figure 5.16). In **NOR flash memory**, the basic unit of access is a bit, referred to as a *memory cell*. Cells in NOR flash are connected in parallel to the bit lines so that each cell can be read/write/erased individually. If any memory cell of the device is turned on by the corresponding word line, the bit line goes low. This is similar in function to a NOR logic gate.<sup>2</sup>

**NAND flash memory** is organized in transistor arrays with 16 or 32 transistors in series. The bit line goes low only if all the transistors in the corresponding word lines are turned on. This is similar in function to a NAND logic gate.

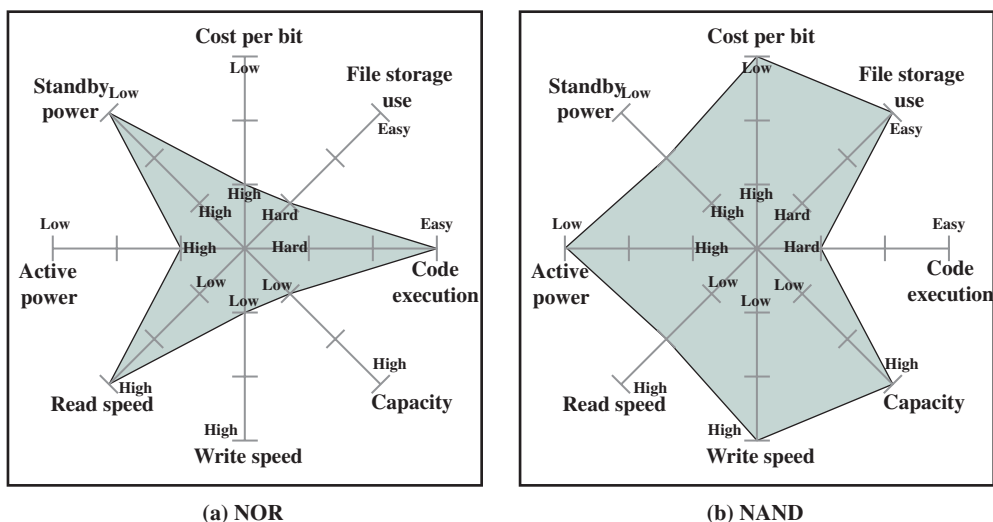
Although the specific quantitative values of various characteristics of NOR and NAND are changing year by year, the relative differences between the two types has remained stable. These differences are usefully illustrated by the Kiviat graphs<sup>3</sup> shown in Figure 5.17.



**Figure 5.16** Flash Memory Structures

<sup>2</sup>The circles associated with and in Figure 5.2b indicate signal negation.

<sup>3</sup>A Kiviat graph provides a pictorial means of comparing systems along multiple variables [MORR74]. The variables are laid out at as lines of equal angular intervals within a circle, each line going from the center of the circle to the circumference. A given system is defined by one point on each line; the closer to the circumference, the better the value. The points are connected to yield a shape that is characteristic of that system. The more area enclosed in the shape, the “better” is the system.



**Figure 5.17** Kiviat Graphs for Flash Memory

NOR flash memory provides high-speed random access. It can read and write data to specific locations, and can reference and retrieve a single byte. NAND reads and writes in small blocks. NAND provides higher bit density than NOR and greater write speed. NAND flash does not provide a random-access external address bus so the data must be read on a blockwise basis (also known as page access), where each block holds hundreds to thousands of bits.

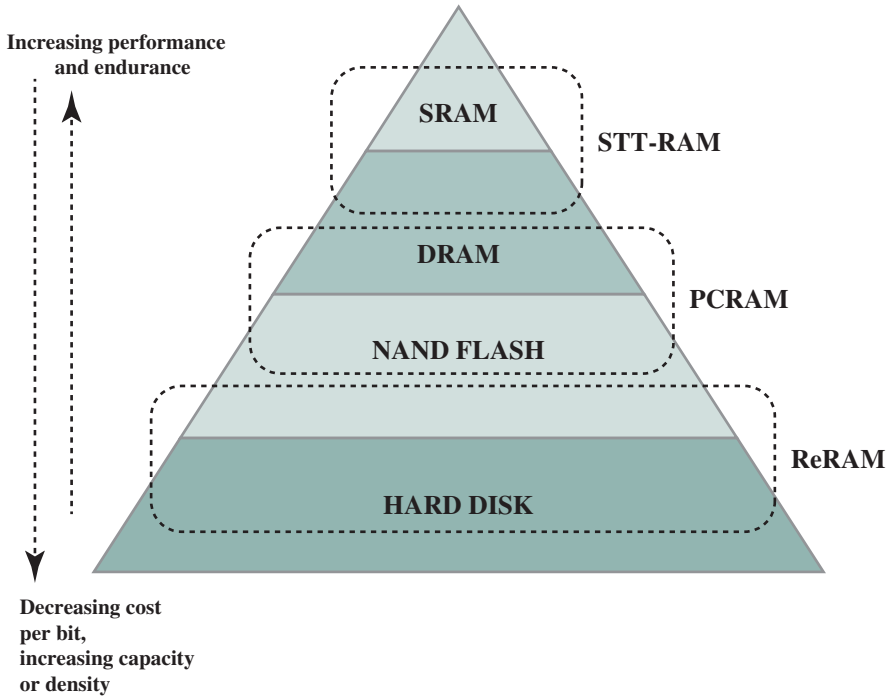
For internal memory in embedded systems, NOR flash memory has traditionally been preferred. NAND memory has made some inroads, but NOR remains the dominant technology for internal memory. It is ideally suited for microcontrollers where the amount of program code is relatively small and a certain amount of application data does not vary. For example, the flash memory in Figure 1.16 is NOR memory.

NAND memory is better suited for external memory, such as USB flash drives, memory cards (in digital cameras, MP3 players, etc.), and in what are known as solid-state disks (SSDs). We discuss SSDs in Chapter 6.

## 5.5 NEWER NONVOLATILE SOLID-STATE MEMORY TECHNOLOGIES

The traditional memory hierarchy has consisted of three levels (Figure 5.18):

- **Static RAM (SRAM):** SRAM provides rapid access time, but is the most expensive and the least dense (bit density). SRAM is suitable for cache memory.
- **Dynamic RAM (DRAM):** Cheaper, denser, and slower than SRAM, DRAM has traditionally been the choice off-chip main memory.
- **Hard disk:** A magnetic disk provides very high bit density and very low cost per bit, with relatively slow access times. It is the traditional choice for external storage as part of the memory hierarchy.



**Figure 5.18** Nonvolatile RAM within the Memory Hierarchy

Into this mix, as we have seen, as been added flash memory. Flash memory has the advantage over traditional memory that it is nonvolatile. NOR flash is best suited to storing programs and static application data in embedded systems, while NAND flash has characteristics intermediate between DRAM and hard disks.

Over time, each of these technologies has seen improvements in scaling: higher bit density, higher speed, lower power consumption, and lower cost. However, for semiconductor memory, it is becoming increasingly difficult to continue the pace of improvement [ITRS14].

Recently, there have been breakthroughs in developing new forms of non-volatile semiconductor memory that continue scaling beyond flash memory. The most promising technologies are spin-transfer torque RAM (STT-RAM), phase-change RAM (PCRAM), and resistive RAM (ReRAM) ([ITRS14], [GOER12]). All of these are in volume production. However, because NAND Flash and to some extent NOR Flash are still dominating the applications, these emerging memories have been used in specialty applications and have not yet fulfilled their original promise to become dominating mainstream high-density nonvolatile memory. This is likely to change in the next few years.

Figure 5.18 shows how these three technologies are likely to fit into the memory hierarchy.

## STT-RAM

STT-RAM is a new type of **magnetic RAM (MRAM)**, which features non-volatility, fast writing/reading speed ( $< 10$  ns), and high programming endurance ( $> 10^{15}$  cycles) and zero standby power [KULT13]. The storage capability or programmability of MRAM arises from magnetic tunneling junction (MTJ), in which a thin tunneling dielectric is sandwiched between two ferromagnetic layers. One ferromagnetic layer (pinned or reference layer) is designed to have its magnetization pinned, while the magnetization of the other layer (free layer) can be flipped by a write event. An MTJ has a low (high) resistance if the magnetizations of the free layer and the pinned layer are parallel (anti-parallel). In first-generation MRAM design, the magnetization of the free layer is changed by the current-induced magnetic field. In STT-RAM, a new write mechanism, called *polarization-current-induced magnetization switching*, is introduced. For STT-RAM, the magnetization of the free layer is flipped by the electrical current directly. Because the current required to switch an MTJ resistance state is proportional to the MTJ cell area, STT-RAM is believed to have a better scaling property than the first-generation MRAM. Figure 5.19a illustrates the general configuration.

STT-RAM is a good candidate for either cache or main memory.

## PCRAM

**Phase-change RAM (PCRAM)** is the most mature of the new technologies, with an extensive technical literature ([RAOU09], [ZHOU09], [LEE10]).

PCRAM technology is based on a chalcogenide alloy material, which is similar to those commonly used in optical storage media (compact discs and digital versatile discs). The data storage capability is achieved from the resistance differences between an amorphous (high-resistance) and a crystalline (low-resistance) phase of the chalcogenide-based material. In SET operation, the phase change material is crystallized by applying an electrical pulse that heats a significant portion of the cell above its crystallization temperature. In RESET operation, a larger electrical current is applied and then abruptly cut off in order to melt and then quench the material, leaving it in the amorphous state. Figure 5.19b illustrates the general configuration.

PCRAM is a good candidate to replace or supplement DRAM for main memory.

## ReRAM

ReRAM (also known as RRAM) works by creating resistance rather than directly storing charge. An electric current is applied to a material, changing the resistance of that material. The resistance state can then be measured and a 1 or 0 is read as the result. Much of the work done on ReRAM to date has focused on finding appropriate materials and measuring the resistance state of the cells. ReRAM designs are low voltage, endurance is far superior to flash memory, and the cells are much smaller—at least in theory. Figure 5.19c shows one ReRAM configuration.

ReRAM is a good candidate to replace or supplement both secondary storage and main memory.

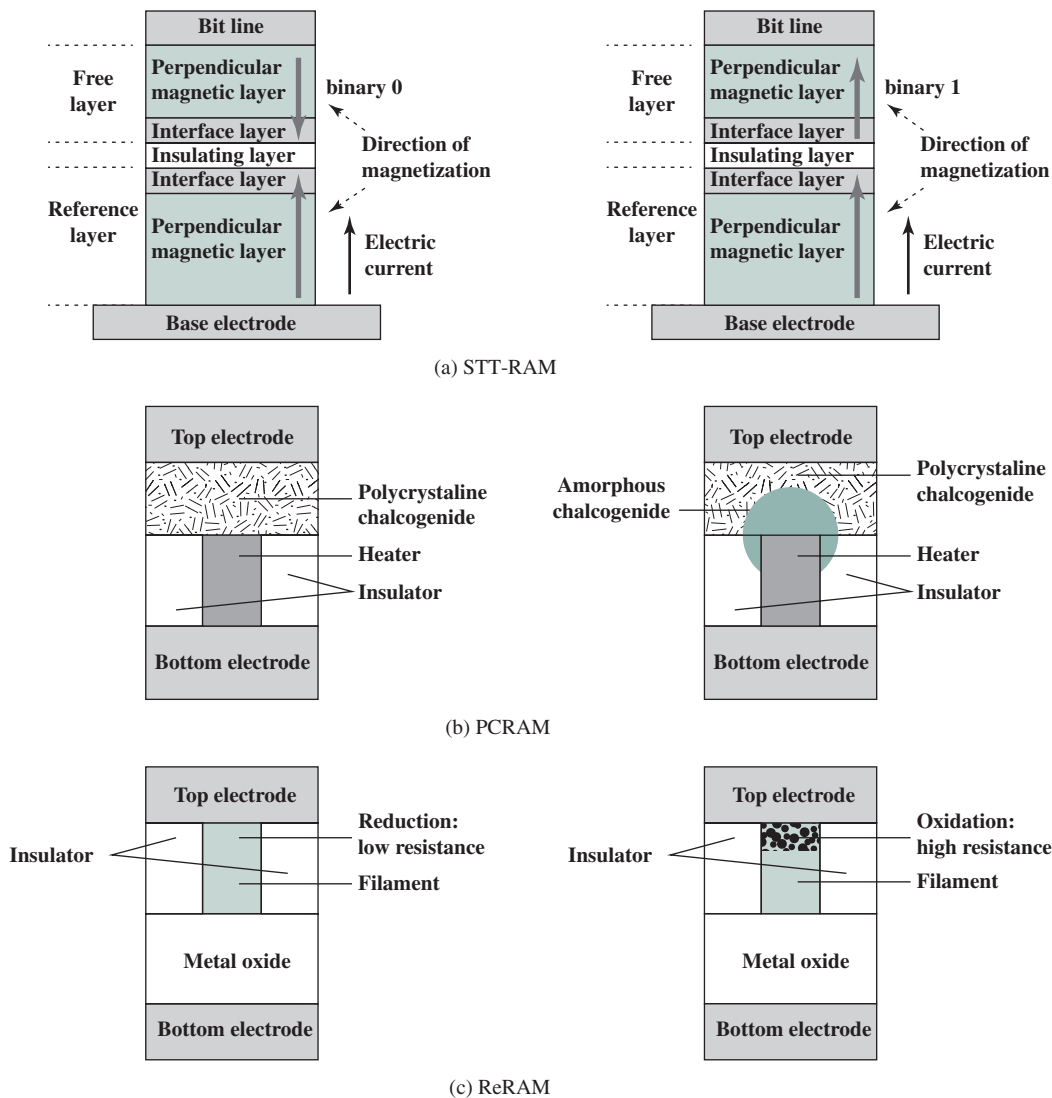


Figure 5.19 Nonvolatile RAM Technologies

5.6 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

|                                  |                                                 |                             |
|----------------------------------|-------------------------------------------------|-----------------------------|
| bank group                       | electrically erasable programmable ROM (EEPROM) | error correcting code (ECC) |
| double data rate DRAM (DDR DRAM) | erasable programmable ROM (EPROM)               | error correction            |
| dynamic RAM (DRAM)               |                                                 | flash memory                |
|                                  |                                                 | Hamming code                |
|                                  |                                                 | hard failure                |

|                                                                                                                                                                                  |                                                                                                                                                                                                                   |                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| magnetic RAM (MRAM)<br>NAND flash memory<br>nonvolatile memory<br>NOR flash memory<br>phase-change RAM<br>(PCRAM)<br>programmable ROM<br>(PROM)<br>random access memory<br>(RAM) | read-mostly memory<br>read-only memory<br>(ROM)<br>resistive RAM (ReRAM)<br>semiconductor memory<br>single-error-correcting<br>(SEC) code<br>single-error-correcting,<br>double-error-detecting<br>(SEC-DED) code | soft error<br>spin-transfer torque RAM<br>(STT-RAM)<br>static RAM (SRAM)<br>synchronous DRAM<br>(SDRAM)<br>syndrome<br>volatile memory |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|

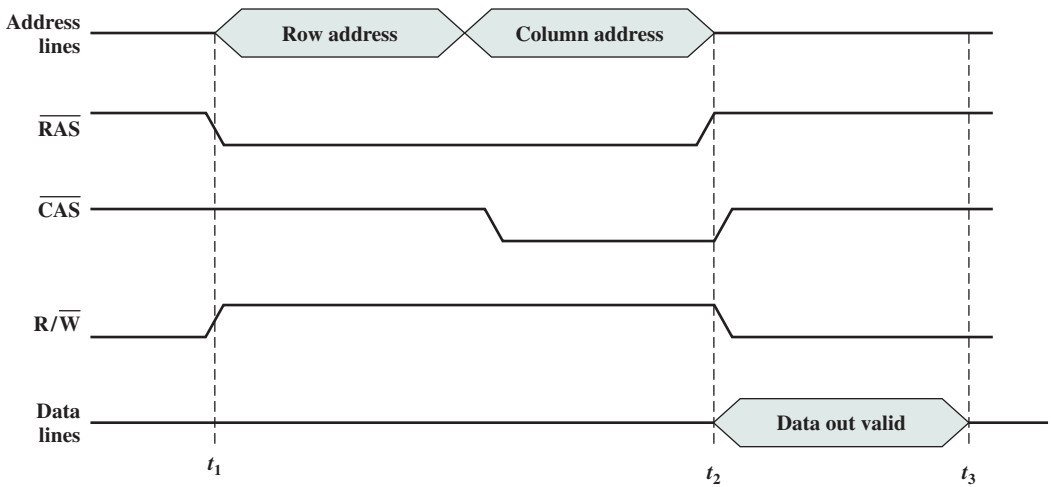
## Review Questions

- 5.1 What are the key properties of semiconductor memory?
- 5.2 What are two interpretations of the term *random-access memory*?
- 5.3 What is the difference between DRAM and SRAM in terms of application?
- 5.4 What is the difference between DRAM and SRAM in terms of characteristics such as speed, size, and cost?
- 5.5 Explain why one type of RAM is considered to be analog and the other digital.
- 5.6 What are some applications for ROM?
- 5.7 What are the differences among EPROM, EEPROM, and flash memory?
- 5.8 Explain the function of each pin in Figure 5.4b.
- 5.9 What is a parity bit?
- 5.10 How is the syndrome for the Hamming code interpreted?
- 5.11 How does SDRAM differ from ordinary DRAM?
- 5.12 What is DDR RAM?
- 5.13 What is the difference between NAND and NOR flash memory?
- 5.14 List and briefly define three newer nonvolatile solid-state memory technologies.

## Problems

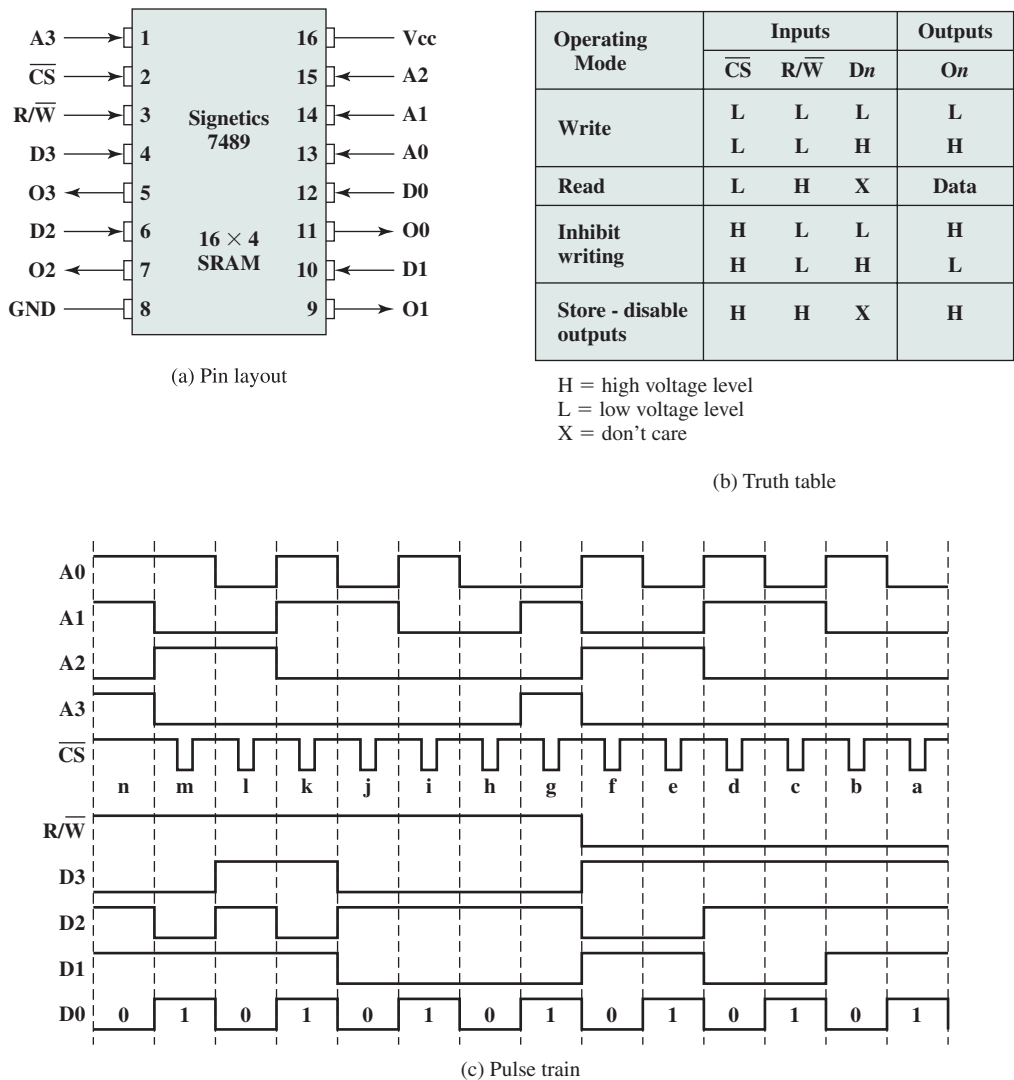
- 5.1 Suggest reasons why RAMs traditionally have been organized as only one bit per chip whereas ROMs are usually organized with multiple bits per chip.
- 5.2 Consider a dynamic RAM that must be given a refresh cycle 64 times per ms. Each refresh operation requires 150 ns; a memory cycle requires 250 ns. What percentage of the memory's total operating time must be given to refreshes?
- 5.3 Figure 5.20 shows a simplified timing diagram for a DRAM read operation over a bus. The access time is considered to last from  $t_1$  to  $t_2$ . Then there is a recharge time, lasting from  $t_2$  to  $t_3$ , during which the DRAM chips will have to recharge before the processor can access them again.
  - a. Assume that the access time is 60 ns and the recharge time is 40 ns. What is the memory cycle time? What is the maximum data rate this DRAM can sustain, assuming a 1-bit output?
  - b. Constructing a 32-bit wide memory system using these chips yields what data transfer rate?
- 5.4 Figure 5.6 indicates how to construct a module of chips that can store 1 MB based on a group of four 256-Kbyte chips. Let's say this module of chips is packaged as a single 1-MB chip, where the word size is 1 byte. Give a high-level chip diagram of how to construct an 8-MB computer memory using eight 1-MB chips. Be sure to show the address lines in your diagram and what the address lines are used for.





**Figure 5.20** Simplified DRAM Read Timing

- 5.5** On a typical Intel 8086-based system, connected via system bus to DRAM memory, for a read operation,  $\overline{\text{RAS}}$  is activated by the trailing edge of the Address Enable signal (Figure C.1 in Appendix C). However, due to propagation and other delays,  $\overline{\text{RAS}}$  does not go active until 50 ns after Address Enable returns to a low. Assume the latter occurs in the middle of the second half of state  $T_1$  (somewhat earlier than in Figure C.1). Data are read by the processor at the end of  $T_3$ . For timely presentation to the processor, however, data must be provided 60 ns earlier by memory. This interval accounts for propagation delays along the data paths (from memory to processor) and processor data hold time requirements. Assume a clocking rate of 10 MHz.
- How fast (access time) should the DRAMs be if no wait states are to be inserted?
  - How many wait states do we have to insert per memory read operation if the access time of the DRAMs is 150 ns?
- 5.6** The memory of a particular microcomputer is built from  $64\text{K} \times 1$  DRAMs. According to the data sheet, the cell array of the DRAM is organized into 256 rows. Each row must be refreshed at least once every 4 ms. Suppose we refresh the memory on a strictly periodic basis.
- What is the time period between successive refresh requests?
  - How long a refresh address counter do we need?
- 5.7** Figure 5.21 shows one of the early SRAMs, the  $16 \times 4$  Signetics 7489 chip, which stores 16 4-bit words.
- List the mode of operation of the chip for each  $\overline{\text{CS}}$  input pulse shown in Figure 5.21c.
  - List the memory contents of word locations 0 through 6 after pulse n.
  - What is the state of the output data leads for the input pulses h through m?
- 5.8** Design a 16-bit memory of total capacity 8192 bits using SRAM chips of size  $64 \times 1$  bit. Give the array configuration of the chips on the memory board showing all required input and output signals for assigning this memory to the lowest address space. The design should allow for both byte and 16-bit word accesses.
- 5.9** A common unit of measure for failure rates of electronic components is the **Failure unit** (FIT), expressed as a rate of failures per billion device hours. Another well known but less used measure is **mean time between failures (MTBF)**, which is the average time of operation of a particular component until it fails. Consider a 1 MB memory of a 16-bit microprocessor with  $256\text{K} \times 1$  DRAMs. Calculate its MTBF assuming 2000 FITs for each DRAM.



**Figure 5.21** The Signetics 7489 SRAM

- 5.10** For the Hamming code shown in Figure 5.10, show what happens when a check bit rather than a data bit is in error?
- 5.11** Suppose an 8-bit data word stored in memory is 11000010. Using the Hamming algorithm, determine what check bits would be stored in memory with the data word. Show how you got your answer.
- 5.12** For the 8-bit word 00111001, the check bits stored with it would be 0111. Suppose when the word is read from memory, the check bits are calculated to be 1101. What is the data word that was read from memory?
- 5.13** How many check bits are needed if the Hamming error correction code is used to detect single bit errors in a 1024-bit data word?
- 5.14** Develop an SEC code for a 16-bit data word. Generate the code for the data word 0101000000111001. Show that the code will correctly identify an error in data bit 5.